CHANDIGARH UNIVERSITY

PROJECT REPORT

ON

"BASINWATCH"

"OFFLINE FLOODPLAIN OPERATIONS CONSOLE"

Submitted in partial fulfilment of the requirements for the subject

OBJECT ORIENTED PROGRAMMING

Subject Code: 25CSH-208

3rd Semester

Bachelor of Engineering (B.E.)

Computer Science Engineering

Submitted By:

Saumya Dwivedi — 25BCS12109

Rishabh Singh — 25BCS11466

Anshu Anand — 25BCS11475

Deepshikha Yadav — 25BCS10801

[ACADEMIC SESSION — TO BE PROVIDED]

[DATE OF SUBMISSION — TO BE PROVIDED]

CERTIFICATE

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"BasinWatch — Offline Floodplain Operations Console"** submitted in partial fulfilment of the requirements for the subject **Object Oriented Programming (25CSH-208)** during the **3rd Semester** of the **Bachelor of Engineering (B.E.) in Computer Science Engineering** at **Chandigarh University** is a bona fide record of the project work carried out by the following students:

S. No.	Student Name	University ID
1	Saumya Dwivedi	25BCS12109
2	Rishabh Singh	25BCS11466
3	Anshu Anand	25BCS11475
4	Deepshikha Yadav	25BCS10801

The work presented in this report is original and has been carried out under the guidance of [SUPERVISOR NAME — TO BE PROVIDED]. To the best of our knowledge, this report has not been submitted to any other university or institution for the award of any degree or diploma.

[CLASS / SECTION / GROUP — TO BE PROVIDED]

_____

[SUPERVISOR NAME — TO BE PROVIDED]

Project Supervisor

_____

[HEAD OF DEPARTMENT — TO BE PROVIDED]

Head of Department

_____

[INTERNAL EXAMINER — TO BE PROVIDED]

Internal Examiner

_____

[EXTERNAL EXAMINER — TO BE PROVIDED]

External Examiner

DECLARATION

STUDENT DECLARATION

We hereby declare that the project report entitled **"BasinWatch — Offline Floodplain Operations Console"**, submitted in partial fulfilment of the requirements for the subject **Object Oriented Programming (25CSH-208)**, 3rd Semester, Bachelor of Engineering (B.E.) in Computer Science Engineering, Chandigarh University, is our own original work.

We further declare that:

- The design, source code, and documentation presented in this report were produced by the team members named below.
- Where external sources have been consulted — including the official Java, Maven, SQLite, and JDBC documentation and standard textbooks — they have been acknowledged in the References section.
- This report, in whole or in part, has not been submitted elsewhere for any academic award.
- The application described is an offline educational planning simulation and is not presented as a real-world flood-forecasting, warning, or emergency-dispatch system.

Student Name	University ID	Signature
Saumya Dwivedi	25BCS12109	
Rishabh Singh	25BCS11466	
Anshu Anand	25BCS11475	
Deepshikha Yadav	25BCS10801	

Place: [TO BE PROVIDED]

Date: [TO BE PROVIDED]

ACKNOWLEDGEMENT

The team records its sincere gratitude to Chandigarh University for providing the academic environment, laboratory access, and library resources that made this project possible. We thank the faculty of the Department of Computer Science Engineering for the structured coverage of object-oriented programming that directly shaped the design of this application.

We are especially grateful to [SUPERVISOR NAME — TO BE PROVIDED], our project supervisor, for guidance on architectural separation, concurrency discipline, and the practice of validating every design claim against running code. The insistence on evidence over assumption influenced both the implementation and this report.

We acknowledge the Head of the Department, [HEAD OF DEPARTMENT — TO BE PROVIDED], and the departmental staff for administrative support and for maintaining the computing facilities used during development and testing. We also thank our classmates and peers who reviewed early demonstrations and offered candid feedback on the usability of the operations console.

Finally, we thank our families for their patience and encouragement during the weeks of design, coding, and documentation. Any errors that remain in the software or in this report are our own.

Saumya Dwivedi — 25BCS12109

Rishabh Singh — 25BCS11466

Anshu Anand — 25BCS11475

Deepshikha Yadav — 25BCS10801

ABSTRACT

Floodplain operations present a genuine coordination problem: rainfall and water-level observations keep arriving while several threatened districts compete for a small, shared stock of pumps, crews, and evacuation vehicles. BasinWatch is an offline floodplain operations console that reproduces this coordination problem as a local desktop planning simulation. It is deliberately **not** a flood-forecasting service, a public warning system, or an emergency dispatch platform; it is an educational simulator whose purpose is to demonstrate, in a single runnable application, how object-oriented design and Java concurrency can keep a live operational picture coherent while independent work proceeds.

The application is written in Java 17 and built with Maven. The user interface uses standard Swing and is owned exclusively by the event-dispatch thread. Beneath the interface, a BasinEngine coordinates a small set of background services. A dedicated SensorFeed thread emits encoded rainfall and water-level observations through connected piped character streams; a SensorIntake task reads, parses, and validates each record and places accepted events into a bounded, monitor-protected event buffer. A small pool of response workers consumes events and applies them to a BasinModel that owns six schematic flood zones, the resource inventory, and active response missions. Immutable snapshots are published to the Swing event queue so that the map, inventory board, and activity timeline always reflect one consistent state.

The project deliberately exercises a broad portion of the object-oriented syllabus. Encapsulation keeps zone and inventory state valid; an abstract event contract and typed event records provide polymorphism at the processing boundary; enums give the domain a fixed vocabulary of risks, missions, resources, and statuses; composition binds zones, missions, and the resource allocator into an aggregate root; and immutable records make it safe to hand data across thread boundaries. Concurrency is treated as a first-class concern: wait() and notifyAll() coordinate a bounded producer-consumer buffer, a short synchronized critical section makes multi-resource reservation all-or-nothing, and shutdown interrupts and joins owned threads with a bounded timeout rather than abandoning file writes.

Persistence is equally deliberate. Operational events are appended to a UTF-8 text journal through a buffered writer; typed sensor measurements are stored in a versioned binary archive written with a data stream over a buffered byte stream; a versioned session snapshot

is serialized to a temporary file and atomically moved into place, with an input filter that rejects unexpected classes; readable situation reports are generated as text; and an optional embedded SQLite database, accessed through raw JDBC with prepared statements and transactions, retains a queryable history without becoming a dependency for the simulation.

The implementation is supported by a dependency-free assertion suite that was executed during preparation of this report and reported seventy-six passing assertions. The suite stresses atomic concurrent resource reservation, bounded-buffer waiting and closing, mission progression and release, session save and restore with corrupt-input rejection, archive validation, JDBC history and transactions, and continued operation when the database is unavailable. The report documents the architecture, the object-oriented design, the concurrency model, the persistence and database layers, the user interface, and the testing approach, and it states clearly the limits of the simulation: the map is schematic, the parameters are illustrative rather than calibrated, and the application controls no real infrastructure.

Keywords: Object-Oriented Programming; Java 17; Java Swing; Multithreading; Producer-Consumer; wait/notifyAll; Synchronization; Resource Allocation; Serialization; Java I/O Streams; JDBC; SQLite; Immutable Snapshots; Offline Simulation; Maven.

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
AWT	Abstract Window Toolkit
B.E.	Bachelor of Engineering
CPU	Central Processing Unit
CSE	Computer Science Engineering
CSV	Comma-Separated Values
DB	Database
DFD	Data-Flow Diagram
DOCX	Microsoft Word Open XML Document
EDT	Event-Dispatch Thread (Swing)
ER	Entity-Relationship
GUI	Graphical User Interface
HOD	Head of Department
IDE	Integrated Development Environment
I/O	Input/Output
JDBC	Java Database Connectivity
JDK	Java Development Kit
JRE	Java Runtime Environment
JVM	Java Virtual Machine
LOC	Lines of Code
Maven	Apache Maven build tool
OOP	Object-Oriented Programming
ORM	Object-Relational Mapping
PRAGMA	SQLite configuration directive
SQL	Structured Query Language
SQLite	Embedded relational database engine
TC / T	Test Case
UC	Use Case
UI	User Interface
UID	University Identification Number
UML	Unified Modeling Language
UTF	Unicode Transformation Format

XML	Extensible Markup Language

TABLE OF CONTENTS

BONAFIDE CERTIFICATE 2
STUDENT DECLARATION 3
ACKNOWLEDGEMENT 5
ABSTRACT 6
LIST OF ABBREVIATIONS 8
TABLE OF CONTENTS ..... None
LIST OF FIGURES ..... None
LIST OF TABLES ..... None
CHAPTER 1: INTRODUCTION 17
1.1 Background 17
1.2 Problem Statement 18
1.3 Motivation 18
1.4 Project Overview 19
1.5 Objectives 20
1.6 Scope 21
1.7 Target Users 21
1.8 Problem-to-Solution Mapping 22
1.9 Limitations 23
1.10 Organization of the Report 23
CHAPTER 2: REQUIREMENTS AND SYSTEM ANALYSIS 24
2.1 Existing Problem 24
2.2 Proposed Solution 24
2.3 Functional Requirements 24
2.4 Non-Functional Requirements 26
2.5 Hardware and Software Requirements 26
2.6 User Roles 27
2.7 Use Cases 27
2.8 Constraints 29
2.9 Feasibility 29
2.10 Assumptions 29
CHAPTER 3: SYSTEM ARCHITECTURE AND DESIGN 30
3.1 Overall Architecture 30
3.2 High-Level System Architecture 30
3.3 Layer and Package Architecture 31
3.4 Component Responsibilities 32
3.5 Data Flow 33
3.6 Runtime Workflow 34

3.7 Class Relationships....36
3.8 Concurrency Architecture....37
3.9 Persistence Architecture....38
3.10 Database Architecture....39
3.11 Error Boundaries....40
3.12 Security and Safe Persistence Considerations....40
3.13 Design Decisions and Rationale....41
CHAPTER 4: OBJECT ORIENTED DESIGN....42
4.1 OOP Principles Used....42
4.2 Encapsulation....46
4.3 Abstraction....47
4.4 Inheritance....48
4.5 Polymorphism....49
4.6 Composition....50
4.7 Association and Aggregation....51
4.8 Constructors....51
4.9 Access Modifiers....52
4.10 Interfaces and the Deliberate Absence of Abstract Classes....53
4.11 Enums....53
4.12 Immutable Objects....54
4.13 Event-Based Design....55
4.14 Exception Handling....55
4.15 OOP Design Benefits....56
CHAPTER 5: JAVA I/O AND PERSISTENCE....57
5.1 Java I/O Overview....57
5.2 Byte Streams....57
5.3 Character Streams....57
5.4 Buffered Streams....57
5.5 Data Streams....58
5.6 Filter and Wrapper Streams....58
5.7 Byte-Array Streams....59
5.8 File and File Metadata....59
5.9 Serialization....59
5.10 Deserialization....60
5.11 `transient` and Runtime-Only State....60
5.12 Session Snapshot....61
5.13 Sensor Archive....61
5.14 Operational Journal....61
5.15 Report Generation....62

5.16 Save and Restore Workflow....62
CHAPTER 6: MULTITHREADING AND CONCURRENCY....64
6.1 Need for Concurrency....64
6.2 Process Versus Thread....64
6.3 Thread Lifecycle....64
6.4 Thread....65
6.5 Runnable....66
6.6 Sensor Feed....67
6.7 Sensor Intake....67
6.8 Event Buffer....67
6.9 Producer-Consumer Pattern....68
6.10 `wait()`....68
6.11 `notifyAll()`....69
6.12 Synchronization....69
6.13 Monitor Locks....69
6.14 Resource Reservation....70
6.15 Atomic Resource Allocation....71
6.16 Concurrent Missions....72
6.17 Deadlock Risk....72
6.18 Deadlock Prevention....72
6.19 Thread Interruption....72
6.20 Graceful Shutdown....73
6.21 Thread Priority and Experimental Diagnostics....74
6.22 Concurrency Safety....75
CHAPTER 7: DATABASE AND JDBC....76
7.1 Need for Database History....76
7.2 SQLite....76
7.3 JDBC Architecture....76
7.4 Connection....78
7.5 PreparedStatement....78
7.6 ResultSet....79
7.7 SQLException....79
7.8 Transactions....79
7.9 Database Schema....79
7.10 Repository Layer....80
7.11 Database Service....80
7.12 Asynchronous Database Operations....81
7.13 Database Failure Handling....81
CHAPTER 8: USER INTERFACE....82

8.1 Swing Architecture....82
8.2 Event Dispatch Thread....82
8.3 Main Operations Console....82
8.4 Basin Map....84
8.5 Systems Inspector....85
8.6 Response Desk....86
8.7 Activity Feed....87
8.8 Database History....87
8.9 User Interaction Flow....88
8.10 UI State Updates....89
8.11 Error Presentation....90
8.12 Screenshots....90
CHAPTER 9: IMPLEMENTATION....92
9.1 Project Structure....92
9.2 Application Startup....92
9.3 Engine Lifecycle....93
9.4 Sensor Feed Implementation....93
9.5 Sensor Intake....93
9.6 Event Processing....93
9.7 Basin Model....94
9.8 Resource Allocation....95
9.9 Mission Processing....95
9.10 Persistence....96
9.11 Database....96
9.12 Report Writer....96
9.13 Session Restore....96
9.14 Error Handling....97
9.15 Important Code Excerpts....97
CHAPTER 10: TESTING AND VALIDATION....98
10.1 Testing Strategy....98
10.2 Unit and Invariant Tests....98
10.3 Concurrency Tests....98
10.4 Resource Contention Tests....99
10.5 Buffer Tests....99
10.6 Persistence Tests....99
10.7 Database Tests....99
10.8 Session Round-Trip Tests....100
10.9 Failure Tests....100
10.10 Shutdown Tests....100

10.11 UI Testing..... 100
10.12 Test Case Table..... 100
10.13 Results..... 103
10.14 Observed Limitations..... 103
CHAPTER 11: RESULTS AND DISCUSSION..... 104
11.1 Functional Results..... 104
11.2 Runtime Workflow..... 104
11.3 Resource Allocation Results..... 105
11.4 Persistence Results..... 105
11.5 Database Results..... 106
11.6 Concurrency Results..... 106
11.7 OOP Learning Outcomes..... 106
11.8 Requirement Achievement..... 106
11.9 Discussion..... 107
CHAPTER 12: CONCLUSION AND FUTURE SCOPE..... 109
12.1 Conclusion..... 109
12.2 Achievements..... 109
12.3 Current Limitations..... 109
12.4 Future Improvements..... 110
12.5 Possible Extensions..... 110
REFERENCES..... 111
APPENDIX A — IMPORTANT SOURCE CODE..... 112
APPENDIX B — TEST RESULTS..... 112
APPENDIX C — DATABASE SCHEMA..... 113
APPENDIX D — CONFIGURATION AND BUILD INFORMATION..... 113
APPENDIX E — ADDITIONAL SCREENSHOTS..... 114
APPENDIX F — ADDITIONAL DIAGRAMS..... 114

LIST OF FIGURES

Figure 1.1: BasinWatch system context and boundary, showing the operator, the offline application, and the local file and database stores....20
Figure 2.1: BasinWatch use-case diagram for the single operator role....28
Figure 3.1: BasinWatch high-level system architecture....30
Figure 3.2: Package diagram for the six BasinWatch packages....31
Figure 3.3: Level-one data-flow diagram....34
Figure 3.4: BasinWatch runtime workflow....35
Figure 3.5: Domain class diagram....37
Figure 3.6: Engine and UI class diagram....37
Figure 3.7: Thread and concurrency map....38
Figure 3.8: Persistence architecture....39
Figure 3.9: SQLite database entity-relationship diagram....39
Figure 4.1: Mapping of object-oriented concepts to BasinWatch components....42
Figure 5.1: Session save and restore flow....63
Figure 6.1: Java thread lifecycle as used by BasinWatch....65
Figure 6.5: Sequence diagram of the sensor and event pipeline....67
Figure 6.2: The bounded producer-consumer event buffer....68
Figure 6.3: Atomic resource reservation and release....71
Figure 6.4: Application lifecycle from startup to clean close....74
Figure 7.1: JDBC architecture of the BasinWatch database layer....77
Figure 8.1: The main BasinWatch console at startup, with observations paused....83
Figure 8.2: The console during a live simulation with active response missions....85
Figure 8.8: The systems inspector showing background thread states and queue metrics....86
Figure 8.3: Sequence diagram of resource dispatch and mission completion....87
Figure 8.4: The database history tab displaying recent records....88
Figure 8.5: Activity diagram of the operator workflow....89
Figure 8.6: The Operations tab listing active and completed field missions....90
Figure 8.7: The Learn tab presenting the in-application architecture notes....91
Figure 11.1: The basin map and operations timeline during a live run....105

LIST OF TABLES

Table 1.1: Problem-to-solution mapping....23
Table 2.1: Functional requirements....25
Table 2.2: Non-functional requirements....26
Table 2.3: Hardware and software requirements....27
Table 2.4: Use cases....28
Table 3.1: Package responsibilities....32
Table 3.2: Component responsibilities and threading....33
Table 3.3: Error boundaries....40
Table 3.4: Design decisions and rationale....41
Table 4.1: Master mapping of object-oriented and Java concepts to the BasinWatch implementation....46
Table 5.1: Persistence artefacts and their stream chains....59
Table 6.1: Event buffer operations and their conditions....68
Table 6.2: Response resource catalogue and standard stock....70
Table 6.3: Mission types, durations, effects, and resource requirements....70
Table 7.1: JDBC concepts and their BasinWatch implementations....77
Table 7.2: SQLite schema....80
Table 7.3: Repository responsibilities....80
Table 8.1: Swing UI components....83
Table 8.2: Console tabs and their purpose....84
Table 8.3: Operator controls....84
Table 9.1: Application startup sequence....93
Table 9.2: Mission processing steps....96
Table 10.1: Test-case summary. Results reflect the executed run of the invariant suite....102
Table 10.2: Executed evidence summary. The suite reported seventy-six passing assertions in total. 103
Table 11.1: Requirement achievement. Status reflects verified functionality, not measured performance....107
Table 11.2: Verified runtime observations and their evidence....107
Table 12.1: Possible future extensions....110

CHAPTER 1: INTRODUCTION

1.1 Background

River basins concentrate people, farmland, and infrastructure on low-lying ground that is periodically inundated. When rainfall intensifies, water levels rise unevenly across a basin: an upstream weir, a densely populated market quarter, and a downstream marsh may each cross a different danger threshold at a different moment. Operations teams respond by reading station observations, ranking the threatened districts, and committing a limited set of physical resources — portable pumps, levee crews, supply trucks, and evacuation vehicles — to the districts that need them most. The difficulty is not a single calculation; it is sustained coordination under change. New observations continue to arrive while earlier decisions are still being carried out, and several districts may be competing for the same scarce equipment at the same time.

Software that supports this kind of work therefore has two obligations that pull in different directions. It must keep updating its picture of the world as fresh data arrives, and it must keep that picture coherent enough that a decision made on it is trustworthy. A program that freezes while it processes an observation is useless to an operator; a program that lets two operations claim the same pump is worse than useless, because it produces confident but false information. Reconciling responsiveness with consistency is precisely the problem that threads and synchronization are designed to solve, which is why this project uses a floodplain operations scenario as the setting for a study of object-oriented programming and Java concurrency.

It is important to state the boundary of the project at the outset. BasinWatch is an offline, single-user, desktop planning simulation. It is an educational artefact built to demonstrate programming concepts; it is not a hydrological model, not a certified forecasting tool, and not a real dispatch system. Its rainfall values, water levels, equipment capacities, and risk thresholds are transparent scenario parameters chosen to make the simulation instructive, not calibrated constants derived from a real basin. This report returns to that boundary repeatedly, because an honest description of what a program is *not* is part of a serious engineering account.

1.2 Problem Statement

The problem that BasinWatch addresses can be stated in one paragraph. *A basin operations team must interpret a continuously arriving stream of rainfall and water-level observations, identify which districts are becoming dangerous, and coordinate a limited inventory of response resources across several districts that may need those resources simultaneously — all without the software either freezing under load or corrupting its own shared state.* Three distinct sub-problems follow from this statement.

The first is **data-arrival concurrency**. Observations are produced on a cadence that is independent of how quickly the operator reads the screen and independent of how quickly earlier observations have been processed. If the arrival of data is handled on the same thread that paints the interface, the interface stutters; if it is handled without a bounded queue, memory can grow without limit; if the queue is unbounded and consumers are slow, the backlog is invisible.

The second is **shared-state integrity**. Zones, resource stock, and mission progress are genuinely shared: several background workers may touch them, and the interface reads them. Two dispatches that run at the same moment must not both observe the same final pump as available. A partially applied change — a mission recorded but its equipment not yet deducted — must never be visible.

The third is **durability and recovery**. An operations picture that evaporates when the window closes is not useful for planning. The system must retain a readable audit of what happened, preserve typed measurements for later review, allow a session to be saved and reopened, and generate a report that a human can read after the simulation ends.

Each sub-problem maps naturally onto object-oriented concepts. Concurrency maps onto threads, monitors, and the producer-consumer pattern. Integrity maps onto encapsulation, composition, and synchronized critical sections. Durability maps onto streams, serialization, and JDBC. This is the sense in which the project is an object-oriented programming project rather than merely a program that happens to be written in Java: the syllabus concepts are the instruments used to solve the domain's problems, not decorations added afterwards.

1.3 Motivation

Three motivations shaped the choice of this project.

First, **a domain that makes concurrency unavoidable**. It is possible to build a Java application in which threads are decorative. BasinWatch is not such an application. The simulation is inherently concurrent: observations arrive on their own schedule while the operator dispatches work and while previously dispatched work progresses. Removing threads would not simplify the design; it would break the behaviour the design is meant to exhibit. The project therefore demonstrates Thread, Runnable, wait(), notifyAll(), synchronization, and interruption because the scenario genuinely needs them.

Second, **the value of a bounded, offline artefact**. An offline desktop application can be built and understood end to end with nothing more than a JDK and Maven. There is no server to install, no network dependency, and no external service whose behaviour must be guessed. This makes every claim in this report verifiable: the reader can compile the project, run its tests, and observe the interface. The optional SQLite history uses an embedded database precisely so that the "database" chapter is real without introducing an operational dependency.

Third, **the opportunity to connect the syllabus to something whole**. Object-oriented courses often teach concepts through small isolated exercises: a class hierarchy here, a thread example there. Building a single application that must simultaneously respect encapsulation, use polymorphism meaningfully, coordinate threads safely, persist data through several stream families, and talk to a relational database through JDBC forces the student to confront how these ideas interact. The result is a richer understanding than any single exercise can provide.

1.4 Project Overview

BasinWatch presents a schematic basin of six monitored zones and lets an operator watch conditions evolve, dispatch responses, and review the consequences. The application is organised into six packages: app for the Swing user interface, domain for the model and its values, engine for the simulation's lifecycle and threading, io for file persistence and reporting, db for the optional SQLite history, and learning for short in-application architecture notes.

At runtime the operator can start a live observation feed or advance the simulation one interval at a time; inject a deliberately marked storm cell into a selected zone; select any zone and read its water level, latest rainfall, exposed population, and risk category; dispatch one of

three response missions; follow the progress of active missions; inspect background service health; browse a typed database history; save a session, close the application, and restore that session later; and export a readable situation report.

Behind the interface, the engine runs a small, deliberately chosen set of background services. One producer thread emits sensor observations. One intake task reads and validates them. A bounded event buffer decouples production from consumption. Three response workers apply events to the model and advance missions. An archive worker writes typed measurements to a binary file, and a single-threaded database service writes history through JDBC. The Swing event-dispatch thread never blocks on this work; it receives immutable snapshots and repaints.

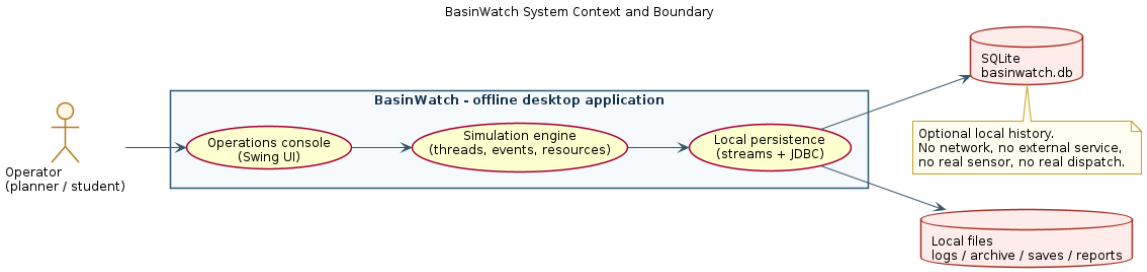


Figure 1.1: BasinWatch system context and boundary, showing the operator, the offline application, and the local file and database stores.

1.5 Objectives

The project set the following objectives. Each is written so that its achievement can be checked against the running program rather than asserted in prose.

- **O1 — Concurrent observation pipeline.** Build a producer-consumer pipeline in which a sensor producer, a validating intake task, a bounded event buffer, and response workers operate concurrently without the user interface blocking. *Verified by:* the engine session test, which observes the feed, pipe, and workers processing real events.
- **O2 — Atomic resource allocation.** Ensure that a multi-resource dispatch is all-or-nothing, so two simultaneous dispatches can never reserve the same unit. *Verified by:* the concurrent reservation test, in which twenty-four contenders attempt to reserve the final pump and crew and exactly one succeeds.
- **O3 — Bounded, safe buffering.** Provide a bounded event buffer that applies back-pressure to producers and drains accepted work on shutdown. *Verified by:* the buffer test, which exercises waiting, back-pressure, and close wake-up.

- **O4 — Multi-format persistence.** Implement an operational text journal, a typed binary sensor archive, a versioned session snapshot, and readable reports, each using the stream family appropriate to its purpose. *Verified by:* the persistence and archive tests.
- **O5 — Safe session recovery.** Save a session and restore it so that zone state, inventory, active missions, and recent activity survive a restart, while rejecting corrupt or unexpected files. *Verified by:* the snapshot round-trip test and the engine save/restore test.
- **O6 — Optional relational history.** Retain a queryable history in an embedded database through raw JDBC with prepared statements and transactions, without making the database a dependency for the simulation. *Verified by:* the JDBC test and the database-unavailable test.
- **O7 — Demonstrate the object-oriented syllabus.** Show, through the actual implementation, the use of encapsulation, abstraction, inheritance, polymorphism, composition, enums, immutability, exception handling, collections, threads, synchronization, serialization, and JDBC.

1.6 Scope

In scope. A single-user, offline desktop simulation of a six-zone schematic basin; synthetic sensor observations; three response mission types; a shared, finite resource inventory; a Swing interface; local file persistence; an optional embedded SQLite history; and a dependency-free invariant test suite.

Out of scope. Any connection to real sensors, gauges, or telemetry; any hydrological or hydrodynamic modelling; any real dispatch, warning, or infrastructure control; any networked or multi-user operation; any cloud service; and any claim that simulated values correspond to a real geographic basin. The schematic map is illustrative and is labelled as such inside the application.

1.7 Target Users

The intended users are students and instructors studying object-oriented programming and Java concurrency, and, secondarily, planners who wish to explore the *shape* of a resource-coordination problem without any operational consequence. The application assumes a single local operator. It presents itself explicitly as an offline planning simulation, and its interface carries that label so that no user mistakes it for an operational tool.

1.8 Problem-to-Solution Mapping

Table 1.1 relates each sub-problem from Section 1.2 to the concrete BasinWatch mechanism that addresses it.

#	Sub-problem	BasinWatch mechanism	Where it is implemented
1	Observations arrive independently of the interface	Dedicated producer thread + piped character streams	engine/ SensorFeed, engine/SensorIntake
2	Slow consumption could grow memory without limit	Bounded, monitor-protected event buffer (capacity 96)	engine/ EventBuffer
3	Interface must not freeze under load	Immutable snapshots handed to the EDT	engine/ BasinEngine, app/MainFrame
4	Two dispatches must not claim the same unit	Atomic, synchronized all-or-nothing reservation	domain/ ResourceAllocator
5	Partially applied changes must not be visible	Synchronized model transitions + immutable snapshots	domain/ BasinModel, domain/BasinSnapshot
6	Decisions must be auditable	UTF-8 operational journal	io/ OperationalJournal
7	Typed measurements must survive the run	Versioned binary archive	io/SensorArchive, engine/ArchiveWorker
8	A session must be reopened later	Versioned, filtered object serialization	io/SnapshotStore, io/SessionSnapshot
9	A human must be able to read outcomes	Text situation report	io/ReportWriter
10	History must be queryable	Embedded SQLite via JDBC	db/*

Table 1.1: Problem-to-solution mapping.

1.9 Limitations

The simulation's honesty depends on stating its limits. The basin map is schematic and is not geospatially accurate. Rainfall, water-level rise, mission durations, equipment capacities, and risk thresholds are illustrative scenario parameters, not calibrated hydrology. The application is single-user and local. It does not ingest real sensors, does not issue alerts, and does not control any gate, pump, vehicle, or crew. Java thread priority is treated only as a best-effort scheduling hint and never as a correctness or ordering mechanism. The session save format is private to this application and versioned; only locally created saves should be loaded. Finally, the exact hydrological realism of the scenario is explicitly not a project objective.

1.10 Organization of the Report

Chapter 2 states the requirements and analyses the system. Chapter 3 describes the architecture and design, including the runtime workflow. Chapter 4 examines the object-oriented design concept by concept, with code excerpts. Chapter 5 covers Java I/O and persistence. Chapter 6 treats multithreading and concurrency in depth. Chapter 7 documents the database and JDBC layer. Chapter 8 describes the user interface. Chapter 9 presents the implementation. Chapter 10 documents testing and validation. Chapter 11 reports results and discusses them. Chapter 12 concludes and outlines future scope. References and appendices follow.

CHAPTER 2: REQUIREMENTS AND SYSTEM ANALYSIS

2.1 Existing Problem

In the absence of a dedicated tool, basin coordination is performed with a patchwork of documents, spreadsheets, and verbal updates. Such an approach struggles with three realities of the domain. First, observations and decisions have different rhythms: readings arrive continuously, while decisions are made at human speed, so any tool must buffer the difference. Second, resources are finite and shared, so the same pump or crew must not be promised twice. Third, the reasoning behind a decision is easily lost, so a durable record matters. These realities are not unique to flood operations — they recur wherever a changing situation is observed while limited resources are committed — which is what makes the scenario a good vehicle for studying concurrency and object-oriented design.

2.2 Proposed Solution

BasinWatch proposes a single, self-contained desktop application that models the coordination problem and makes its mechanics visible. The operator watches a schematic basin evolve, dispatches responses from a finite inventory, and reviews the consequences in a timeline. Under the surface, the application separates *producing* observations from *consuming* them with a bounded buffer, protects shared inventory with a short synchronized transaction, and persists its state through several complementary mechanisms. The solution is deliberately bounded: it demonstrates the mechanics faithfully while making no operational claims.

2.3 Functional Requirements

ID	Requirement
FR-1	The system shall present a schematic basin of six monitored zones.
FR-2	Each zone shall expose a water level, latest rainfall, exposed population, and a risk category.
FR-3	The system shall generate synthetic rainfall and water-level observations on a configurable

	interval.
FR-4	The system shall allow the operator to advance the simulation one interval at a time.
FR-5	The system shall allow the operator to inject a marked storm-cell observation into a selected zone.
FR-6	The system shall validate every observation and discard malformed records with a visible warning.
FR-7	The system shall rank zones by risk and highlight escalating risk.
FR-8	The system shall maintain a finite inventory of pumps, levee crews, supply trucks, evacuation vehicles, and field crews.
FR-9	The system shall allow the operator to dispatch pump, levee-reinforcement, and evacuation missions to a selected zone.
FR-10	A dispatch shall reserve all of a mission's required resources atomically or be rejected.
FR-11	Active missions shall advance deterministically with simulation intervals.
FR-12	Completed missions shall return their resources to the shared inventory.
FR-13	The system shall display available and committed units and active operations.
FR-14	The system shall record operational events to a readable journal.
FR-15	The system shall archive typed sensor measurements to a binary file.
FR-16	The system shall save and restore a session, rejecting corrupt or unsupported files.
FR-17	The system shall export a readable situation report.
FR-18	The system shall retain an optional queryable history in an embedded database and shall continue operating when that database is unavailable.

Table 2.1: Functional requirements.

2.4 Non-Functional Requirements

ID	Requirement
NFR-1	The user interface shall remain responsive while observations and operations are processed.
NFR-2	Shared state shall never be observed in a partially applied form.
NFR-3	The event buffer shall be bounded so that memory cannot grow without limit.
NFR-4	Shutdown shall stop new work, drain accepted work, and close files within a bounded time.
NFR-5	Persistence failures shall be reported and shall not corrupt existing state.
NFR-6	A corrupt or unexpected save file shall be rejected with an actionable message.
NFR-7	The database shall be optional; its failure shall not stop the simulation or file persistence.
NFR-8	The application shall run offline with no network dependency.
NFR-9	The build shall be reproducible with Maven and shall bundle its single runtime dependency.
NFR-10	The codebase shall be testable without an external test framework.

Table 2.2: Non-functional requirements.

2.5 Hardware and Software Requirements

Category	Requirement
Processor	Any modern 64-bit CPU (the workload is light and CPU-bound only briefly)
Memory	2 GB or more (the application and its buffers are small)
Display	1100×720 minimum window; 1440×860 typical
Operating system	Windows (primary deployment), Linux or macOS (development)
Runtime	JDK 17 or newer; the packaged Windows app

	image bundles a private runtime
Build tool	Maven 3.8 or newer
Database	None to install — SQLite is embedded
Network	Not required

Table 2.3: Hardware and software requirements.

2.6 User Roles

BasinWatch defines a single role, the **operator**, who observes the basin, selects zones, dispatches responses, and manages sessions. There is no separate administrator role, no authentication, and no multi-user access, because the application is a local single-user simulation. The in-application "Learn" content is available to the same operator and requires no separate privilege.

2.7 Use Cases

ID	Use case	Actor	Summary
UC-1	Start/pause observations	Operator	Begin or suspend the synthetic observation feed
UC-2	Advance one interval	Operator	Emit one interval of observations and advance missions
UC-3	Inject storm cell	Operator	Queue a marked high-rainfall observation for a zone
UC-4	Select a zone	Operator	Choose a zone from the map or the selector
UC-5	Dispatch a response	Operator	Reserve resources and start a mission
UC-6	Review active operations	Operator	Follow mission progress
UC-7	Inspect background activity	Operator	View worker states and queue metrics
UC-8	Save/restore session	Operator	Persist or recover the simulation

UC-9	Export situation report	Operator	Write a readable report
UC-10	View database history	Operator	Browse recent readings and records

UC-5 includes UC-4, because a dispatch requires a selected zone.

Table 2.4: Use cases.

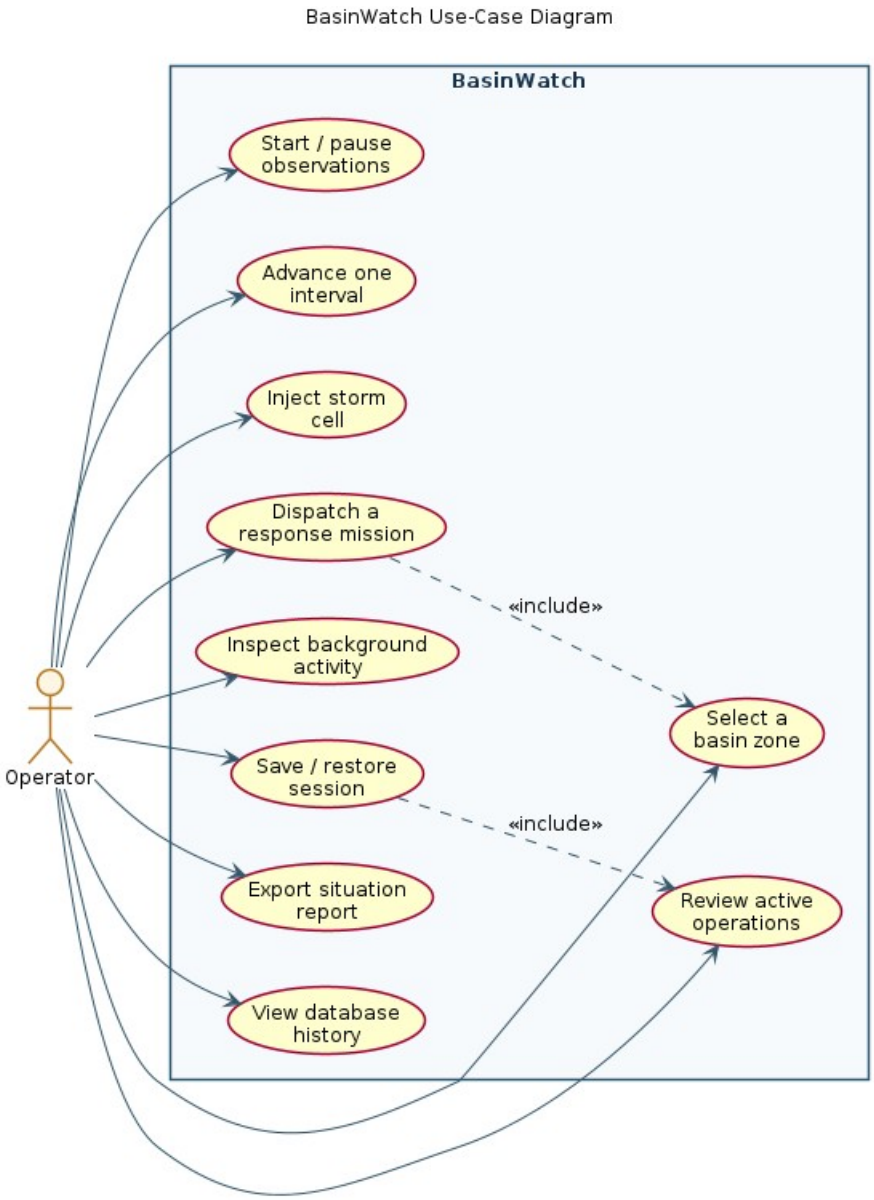


Figure 2.1: BasinWatch use-case diagram for the single operator role.

2.8 Constraints

The project is constrained to the standard JDK and Swing; no JavaFX runtime, web server, or external service is used. The only third-party runtime dependency is the SQLite JDBC driver. The simulation must run offline. The design must not rely on thread priority for correctness. The session format is application-private and versioned. Finally, the project must be demonstrable on a single machine without specialised hardware.

2.9 Feasibility

Technical feasibility. The application uses only standard, well-documented Java facilities: Swing for the interface, java.io for persistence, the language's built-in threading and monitor support for concurrency, and JDBC for the embedded database. Every mechanism used is part of the standard platform, so the project is technically feasible without bespoke infrastructure.

Economic feasibility. The software requires no licences, no server, and no cloud spend. The only cost is development time and the hardware already available to students.

Operational feasibility. Because the application is offline and self-contained, it can be demonstrated and graded on a single machine. Its data lives in a per-user folder and can be relocated with a single command-line option, so it does not interfere with the host system.

2.10 Assumptions

The design assumes that observations are synthetic and that their parameters are illustrative; that a single operator interacts with the application at a time; that the local file system is writable; that the embedded database may be absent or unusable, in which case the simulation must continue; and that thread scheduling is nondeterministic, so correctness must never depend on the order in which threads run.

CHAPTER 3: SYSTEM ARCHITECTURE AND DESIGN

3.1 Overall Architecture

BasinWatch is a single-process, offline desktop application organised into four logical layers. The **presentation layer** (com.basinwatch.app) consists of Swing components that are owned exclusively by the event-dispatch thread (EDT). The **application and engine layer** (com.basinwatch.engine) owns the simulation's lifecycle and its background services. The **domain layer** (com.basinwatch.domain) holds the model of the basin — zones, events, resources, and missions — and enforces its invariants. The **persistence layer** (com.basinwatch.io and com.basinwatch.db) writes and reads operational files and the optional embedded database.

The layering is enforced by direction of dependency rather than by a framework: the presentation layer depends on the engine and domain; the engine depends on the domain and persistence; the domain depends on nothing outside itself and the standard library. This keeps the domain testable in isolation and prevents file or database concerns from leaking into the model.

3.2 High-Level System Architecture

Figure 3.1 shows the four layers and the principal classes in each. The engine is the hub: it owns the sensor feed, intake task, event buffer, response workers, archive worker, journal, snapshot store, report writer, and database service, and it connects them to the domain model.

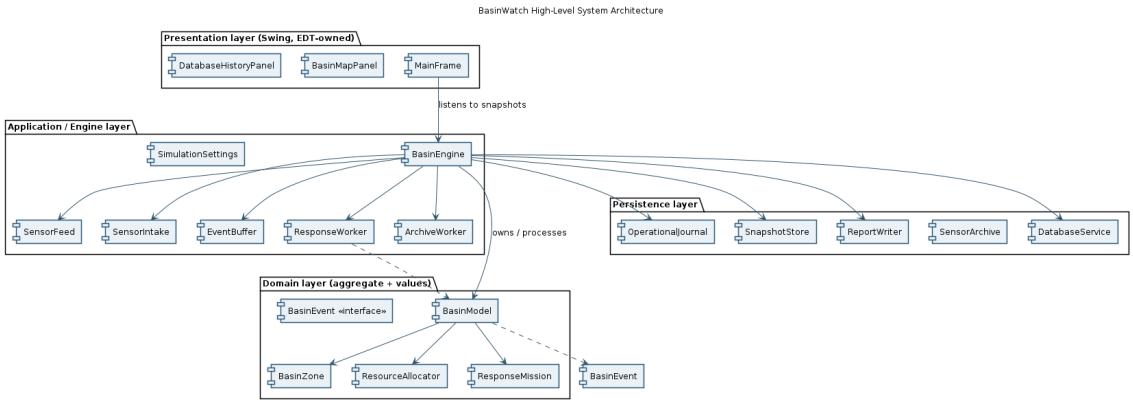


Figure 3.1: BasinWatch high-level system architecture.

3.3 Layer and Package Architecture

The project is divided into six packages. Figure 3.2 shows the package dependency graph, and Table 3.1 summarises each package's responsibility.

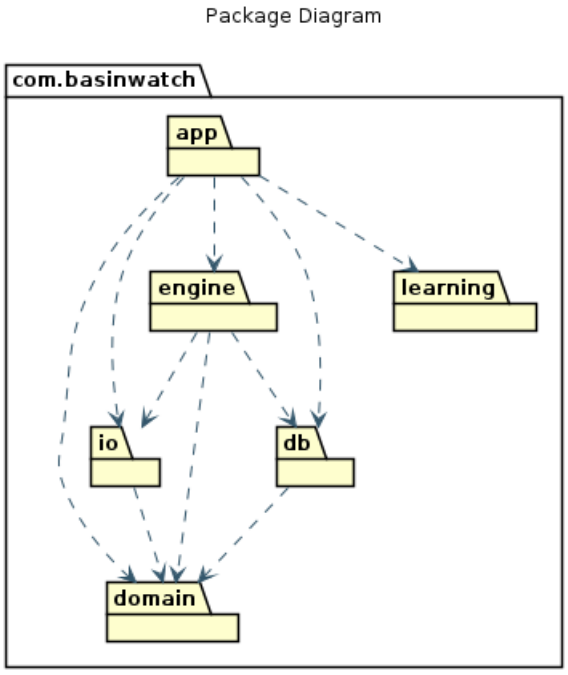


Figure 3.2: Package diagram for the six BasinWatch packages.

Package	Responsibility	Notable types
com.basinwatch.app	Swing entry point, frame, map panel, database history panel	BasinWatchApp, MainFrame, BasinMapPanel, DatabaseHistoryPanel
com.basinwatch.domain	Model, events, resources, missions, risk	BasinModel, BasinZone, BasinEvent, ResourceAllocator, ResponseMission
com.basinwatch.engine	Lifecycle, threading, buffering, workers, diagnostics	BasinEngine, SensorFeed, SensorIntake, EventBuffer, ArchiveWorker
com.basinwatch.io	File persistence, snapshots, archive, reports	AppPaths, SnapshotStore, SensorArchive, OperationalJournal,

		ReportWriter
com.basinwatch.db	Optional embedded SQLite history via JDBC	DatabaseService, DatabaseManager, repositories
com.basinwatch.learning	In-application architecture notes	ArchitectureCard, LearningCatalog

Table 3.1: Package responsibilities.

3.4 Component Responsibilities

Table 3.2 lists the major runtime components, the thread on which each executes, and its responsibility. The threading column is central to the architecture and is examined in depth in Chapter 6.

Component	Runs on	Responsibility
MainFrame and panels	Swing EDT	Render immutable snapshots; capture operator actions
BasinEngine	Caller + basin-control executor	Own lifecycle; start/pause/save/load/report/close
SensorFeed	basin-sensor-feed thread	Emit encoded observations and pulses on an interval
SensorIntake	basin-sensor-intake thread	Read, parse, validate, and enqueue events
EventBuffer	Shared (guarded)	Bounded producer-consumer queue with wait()/notifyAll()
ResponseWorker (×3)	basin-response-1..3 threads	Apply events to the model; advance and complete missions
ArchiveWorker	basin-sensor-archive thread	Write typed measurements to the binary archive
DatabaseService	basin-database thread	Write/read SQLite history off the EDT
BasinModel	Guarded by synchronized	Own zones, inventory, missions, activity; return snapshots
ResourceAllocator	Guarded by synchronized	Atomic all-or-nothing reservation and release

Table 3.2: Component responsibilities and threading.

3.5 Data Flow

Figure 3.3 presents a level-one data-flow view. Observations originate in the sensor feed, are validated at intake, are buffered, are applied to the model by response workers, and result in an immutable snapshot delivered to the interface and in records written to the journal, archive, saves, and database.

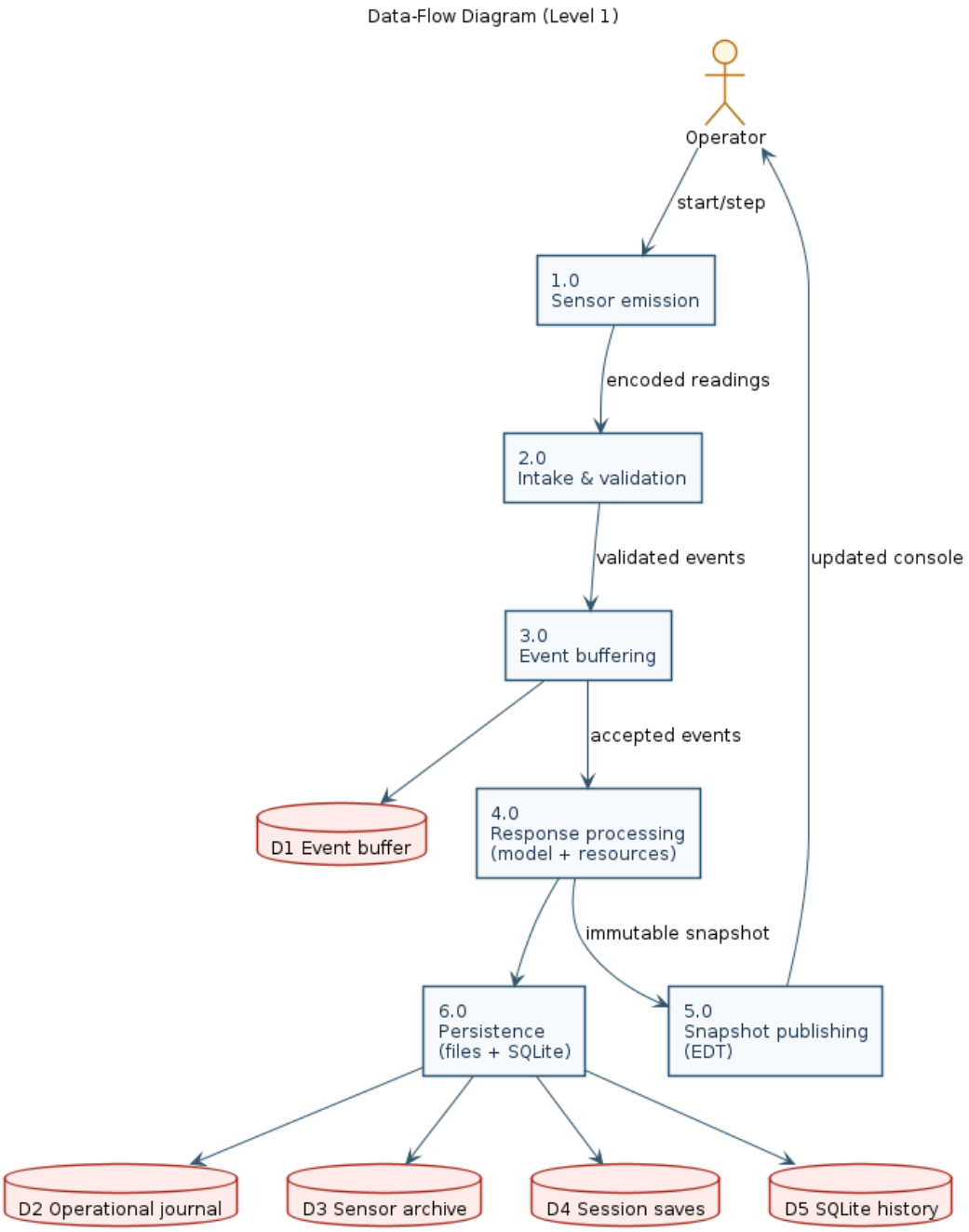


Figure 3.3: Level-one data-flow diagram.

3.6 Runtime Workflow

Figure 3.4 shows the end-to-end runtime workflow. The producer writes encoded records to a PipedWriter; the intake task reads them through a PipedReader wrapped in a BufferedReader; accepted events enter the bounded buffer; response workers take them, apply them to the model under synchronization, and publish an immutable snapshot to the EDT.

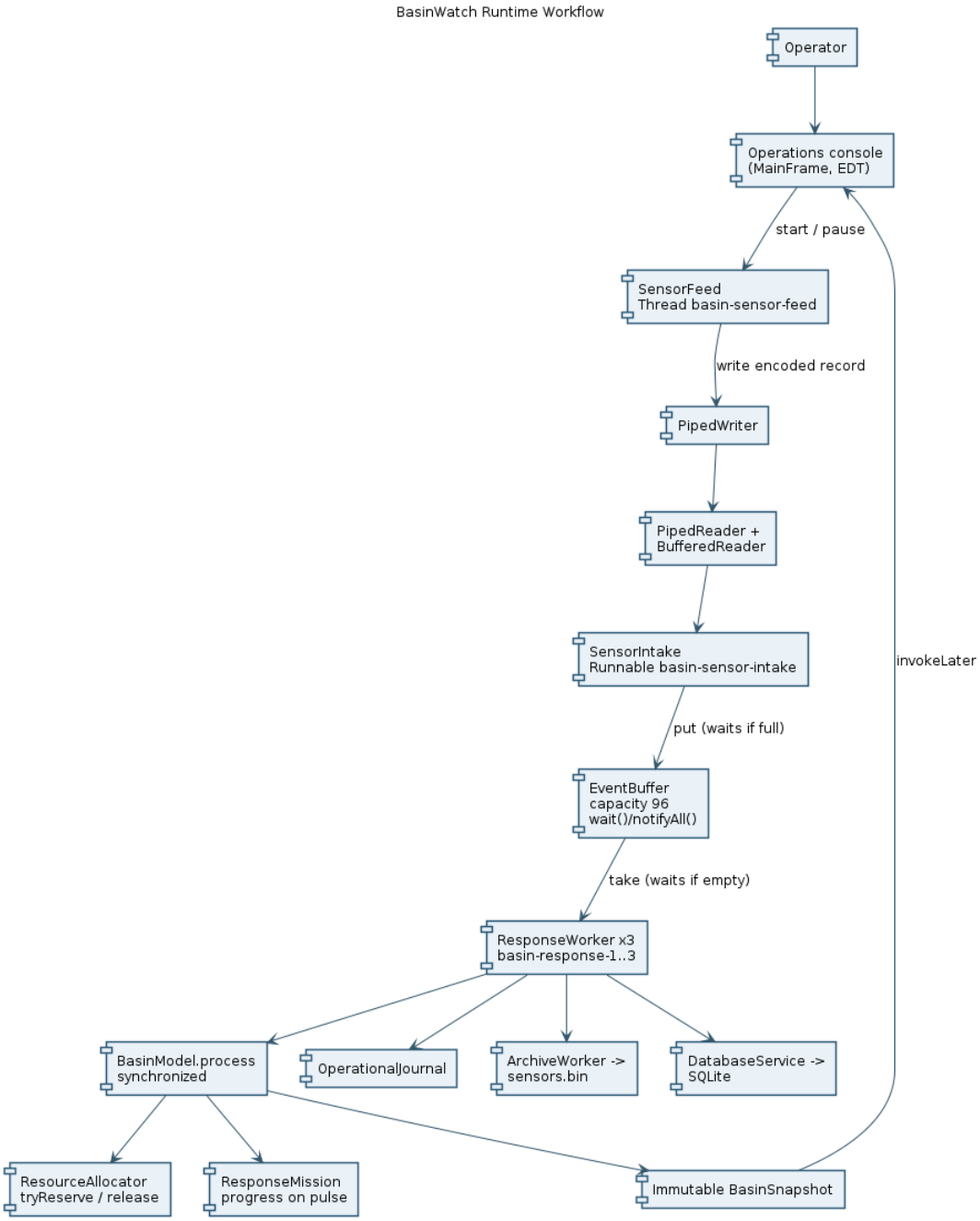


Figure 3.4: BasinWatch runtime workflow.

The engine starts the feed by constructing a connected pair of piped streams and starting the intake thread before the feed thread, so that a reader exists before any bytes are written. Listing 3.1 shows this startup.

```
// engine/BasinEngine.java (startFeed, line 279, condensed)
PipedWriter writer = new PipedWriter();
PipedReader reader = new PipedReader(writer, 8_192);
feedWriter = writer;
intakeThread = new Thread(new SensorIntake(
```

```
        new BufferedReader(reader), eventBuffer, this::reportFailure,
        ignored -> publishDiagnosticsStatus()), "basin-sensor-intake");
sensorFeed = new SensorFeed(writer, List.of("UPR", "MIL", "NOR", "OLD", "MAR", "SOU"),
        settings, tickSequence, this::reportFailure);
intakeThread.start();
sensorFeed.start();
```

Listing 3.1: The engine connects the sensor feed and intake task through piped character streams, starting the reader first.

When an event reaches a worker, the engine processes it inside a synchronized block that captures the model state before and after the transition, so that persistence can reason about what changed. Listing 3.2 shows the core of this method.

```
// engine/BasinEngine.java (process, line 441, condensed)
private void process(BasinEvent event) throws IOException, InterruptedException {
    BasinSnapshot before;
    BasinSnapshot after;
    List<String> messages;
    synchronized (model) {
        before = model.snapshot();
        messages = model.process(event);
        after = model.snapshot();
    }
    // ... journal each message, archive the reading, persist database deltas ...
    publishSnapshot();
}
```

Listing 3.2: Event processing captures the before-and-after model state under synchronization, then performs I/O outside the lock.

Note that the lock is held only while the model transition is computed. Journal writing, archive submission, and database enqueueing happen after the synchronized block, so no blocking I/O is performed while holding the model lock.

3.7 Class Relationships

Figures 3.5 and 3.6 show the domain and the engine/UI class relationships respectively. The domain is an aggregate: BasinModel owns six BasinZone objects, up to five hundred ResponseMission objects, one ResourceAllocator, and a bounded deque of ActivityEntry records. Events implement the BasinEvent interface. The engine composes its services and holds a small list of response-worker threads.

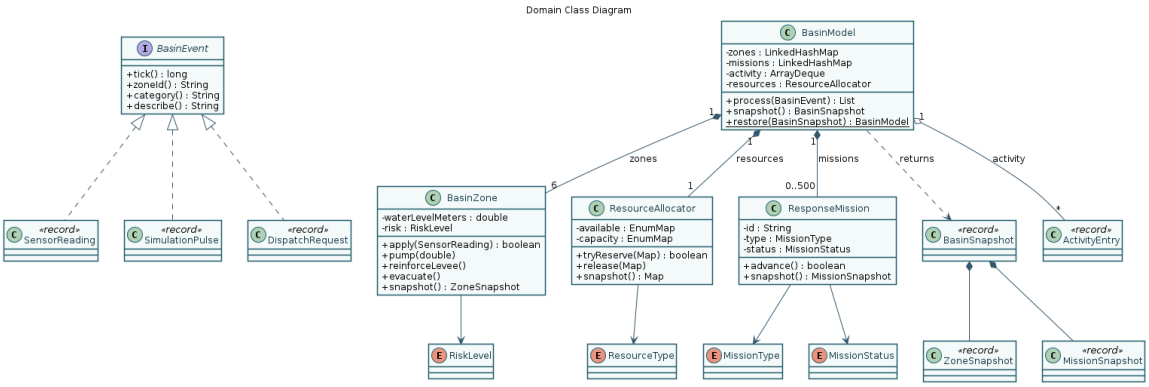


Figure 3.5: Domain class diagram.

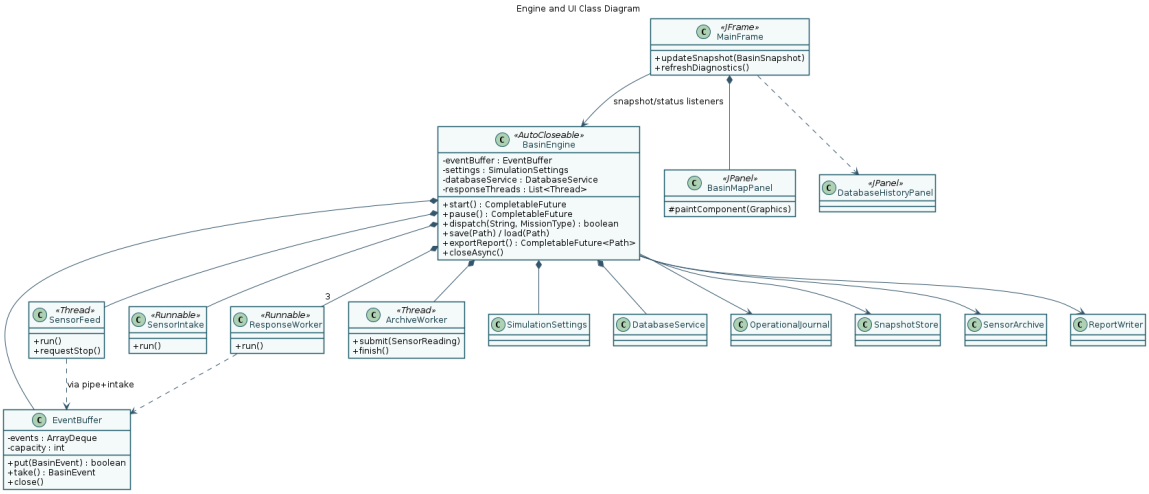


Figure 3.6: Engine and UI class diagram.

3.8 Concurrency Architecture

Concurrency is the architectural spine of the application. Figure 3.7 maps the threads and their interactions. The Swing EDT owns all components. The basin-control daemon executor serialises lifecycle actions such as start, pause, save, load, report, and close. The sensor feed and intake form the production side of the pipeline. Three response workers form the consumption side. The archive worker and database service are separate, low-urgency writers. Immutable snapshots cross from workers to the EDT.

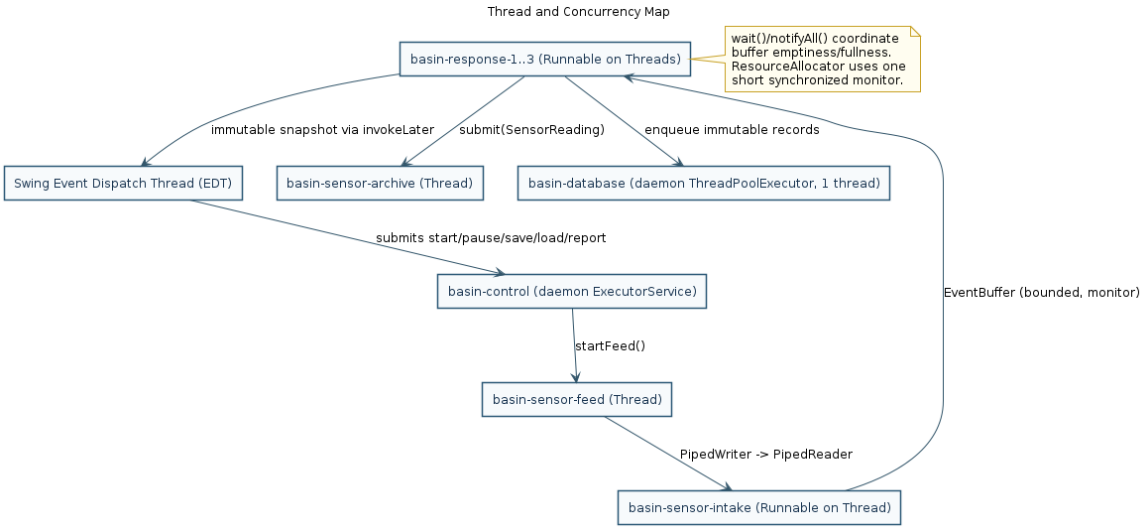


Figure 3.7: Thread and concurrency map.

The architecture deliberately avoids nested locks in the normal path. The event buffer uses its own intrinsic monitor; the resource allocator uses its own; the model uses its own. The allocator's critical section contains only inventory validation and arithmetic — never I/O, never a wait on a different monitor — which removes the circular-wait precondition from normal operation. Section 6.17 examines the deadlock hazard and its prevention in detail.

3.9 Persistence Architecture

Figure 3.8 shows the persistence architecture. Five mechanisms serve five purposes: a UTF-8 text journal for human-readable events; a versioned binary archive for typed measurements; a versioned session snapshot for recovery; UTF-8 text reports for review; and an embedded SQLite database for queryable history. Each mechanism uses the stream or API appropriate to its data, and each is isolated so that a failure in one does not corrupt the others.

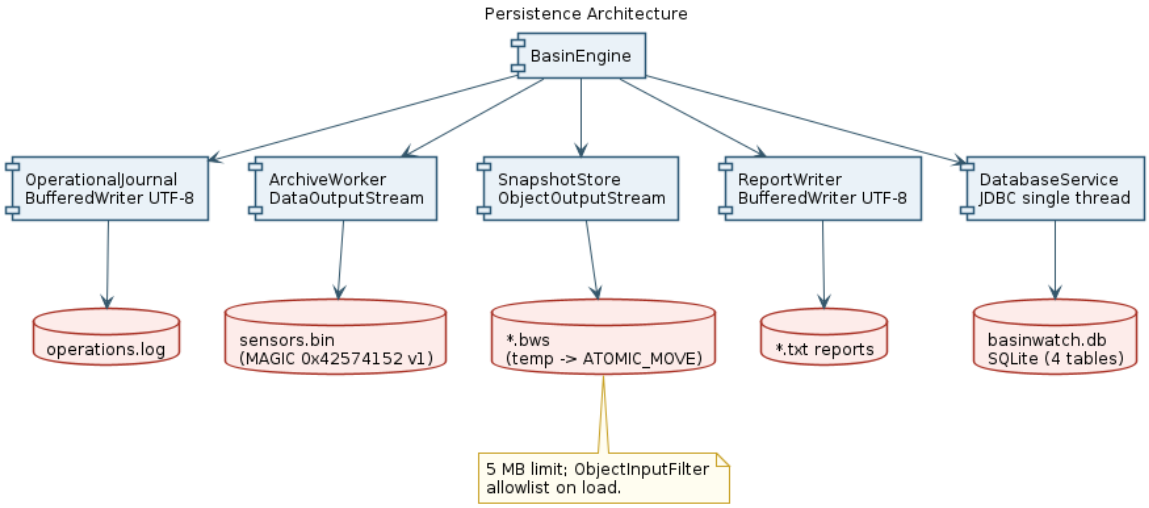


Figure 3.8: Persistence architecture.

3.10 Database Architecture

The optional database layer wraps SQLite behind a single-threaded service. All SQL runs on the basin-database thread, never on the EDT. Repositories open fresh connections, use prepared statements, and close resources with try-with-resources; linked mission records are written in a transaction. Figure 3.9 is the entity-relationship diagram for the four tables, and Chapter 7 documents the layer fully.

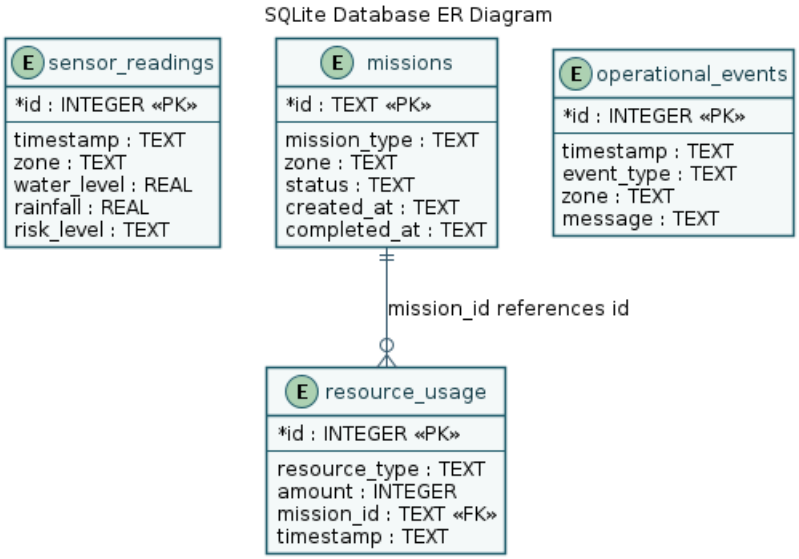


Figure 3.9: SQLite database entity-relationship diagram.

3.11 Error Boundaries

The architecture places a deliberate boundary around each category of failure so that one failure cannot silently become another. Table 3.3 summarises the boundaries.

Boundary	Failure handled	Behaviour
Sensor intake	Malformed record	Log a warning and discard; intake continues
Domain constructors/commands	Impossible zone, negative quantity, invalid transition	Throw `IllegalArgumentException` / `IllegalStateException`
Resource allocator	Insufficient units	Return `false`; record a typed rejection
Snapshot store	Missing, oversized, corrupt, or unexpected class	Throw `IOException` with an actionable message
Sensor archive	Bad marker or version	Throw `IOException` on open
Database service	`SQLException`	Report and isolate; simulation continues
Response worker	Runtime or I/O failure	Report with event context; continue unrelated work
Shutdown	Interrupted or timed-out join	Preserve interrupt status; report, do not pretend success
UI action	Any reported failure	Show a status banner or dialog; keep prior valid state

Table 3.3: Error boundaries.

3.12 Security and Safe Persistence Considerations

Although the application handles no secrets and has no network surface, it treats its own save files as untrusted input. The snapshot store bounds the payload to five megabytes, reads at most one byte beyond that limit, and installs an ObjectInputFilter that rejects any class outside an explicit allowlist and any object graph exceeding shallow depth or reference limits. Because Java deserialization of arbitrary classes is a well-known hazard, restricting it to the application's own snapshot classes is a deliberate safety measure, not an afterthought. The database layer uses prepared statements with bound parameters for all application data, which prevents SQL injection through recorded values.

3.13 Design Decisions and Rationale

Decision	Alternative considered	Rationale
Bounded event buffer (capacity 96)	Unbounded queue	Bounds memory and applies visible back-pressure
Three response workers	One worker or a fixed thread pool per event	Keeps a small, predictable worker set while allowing overlap
Immutable snapshots to the EDT	Let the UI read the live model	Avoids exposing mutable state across threads
One short allocator monitor	Multiple per-resource locks	Removes circular-wait risk and simplifies reasoning
Piped character streams for observations	Direct method calls or a shared queue	Demonstrates the piped-stream mechanism and decouples producer from consumer
Versioned, filtered serialization	Plain serialization or JSON	Safe, self-describing local saves with corrupt-input rejection
Embedded SQLite via raw JDBC	An ORM or an external database server	Keeps the database real, offline, and dependency-free
Text journal separate from binary archive	A single format	Serves human review and typed replay with the right stream each
Priority as an optional experiment	Priority-based scheduling	Priority is nondeterministic and must not affect correctness

Table 3.4: Design decisions and rationale.

CHAPTER 4: OBJECT ORIENTED DESIGN

This chapter is the analytical heart of the report. It does not merely list the object-oriented features that happen to appear in the code; it explains, for each concept that the project genuinely uses, *where* it appears, *why* the domain needs it, *how* it works, and *how* it connects to the syllabus. Where a concept is deliberately not used, that too is stated, because a disciplined design excludes features it does not need.

4.1 OOP Principles Used

The application is built from fifty classes distributed across six packages. Every class has a single, statable responsibility, and the object-oriented mechanisms are chosen to serve the domain rather than to satisfy a checklist. The concepts used are encapsulation, abstraction, inheritance (narrowly), polymorphism, composition, association, aggregation, constructors, access modifiers, interfaces, enums, static members, immutability, event-driven design, exception handling, collections, threads and monitors, the producer-consumer pattern, serialization, file handling, and JDBC. Abstract classes are not used. Figure 4.1 maps each concept to the component that realises it, and Table 4.1 is the master mapping.

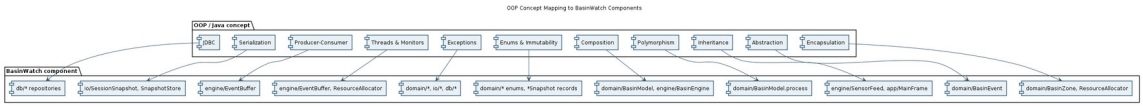


Figure 4.1: Mapping of object-oriented concepts to BasinWatch components.

OOP / Java concept	BasinWatch component	Actual class / file	Purpose	Implementation explanation
Classes and objects	Whole application	50 classes, 6 packages	Model the basin domain	Each class owns its state and behaviour
Encapsulation	Guarded domain state	domain/ BasinZone, domain/Resou rceAllocator, io/SessionSna pshot	Prevent invalid states	Private fields; package-private mutators; validating constructors
Abstraction	Event contract	domain/ BasinEvent	Uniform event handling	Interface with tick, zoneId,

				category, describe
Inheritance	Threads and Swing components	engine/ SensorFeed, engine/ArchiveWorker, app/MainFrame, app/BasinMapPanel	Adopt framework lifecycles	Seven extends sites
Polymorphism	Event, risk, and mission dispatch	domain/ BasinModel, app/MainFrame, db/DatabaseService	Type-specific behaviour	instanceof dispatch; enum switch; overridden methods
Method overriding	26 @Override methods	SensorFeed.run, BasinMapPanel.paintComponent, …	Adapt framework hooks	Override run, paintComponent, toString
Method overloading	Alternate constructors	domain/ ResponseMission, domain/ResourceAllocator	Alternate construction	Two constructors each
Constructors	Validated creation	Records, enums, BasinModel	Enforce invariants at creation	Compact record constructors throw on invalid input
Access modifiers	Least privilege	All packages	Controlled exposure	One protected (paintComponent); private nested interfaces
Composition	Aggregate roots	domain/ BasinModel, engine/BasinEngine	Lifecycle coupling	final fields; aggregate ownership

Association	Engine ↔ database, UI ↔ engine	engine/ BasinEngine, db/DatabaseService	Collaborate without ownership	Listener callbacks and injected references
Aggregation	Shared vocabulary	domain/ MissionType.requirements, engine/SimulationSettings	Shared data	Map<ResourceType,Integer>; shared settings object
Interfaces	Contracts	domain/ BasinEvent and four functional interfaces	Decouple producers and consumers	Public domain interface; private functional interfaces
Abstract classes	Not used	—	Deliberate exclusion	Inheritance only for real framework contracts
Enums	Bounded vocabularies	RiskLevel, MissionType, MissionStatus, ResourceType, DatabaseHistoryType	Safe, readable states	Enum constructors carry labels, ranks, requirements
Static members	Constants and factories	domain/ ResourceAllocator, domain/BasinZone	Shared values and utilities	33 static final constants; standard(), assessRisk()
Immutable objects	14 records	domain/*, engine/ThreadDiagnostic, db/DatabaseHistoryEntry	Safe cross-thread sharing	Records plus List.copyOf/Map.copyOf
Event-driven design	Pipeline and listeners	engine/ BasinEngine	Decouple producer, consumer, UI	Immutable events; listener lists; EDT hand-off
Exception handling	Every boundary	domain/*, io/*, db/*, engine/*	Distinct visible outcomes	Validation, checked I/O,

				SQLException, interrupt preservation
Collections	Storage and queues	domain/*, engine/*, db/*	Order, dedupe, thread-safe queues	LinkedHashMap, ArrayDeque, EnumMap, ArrayBlockingQueue, CopyOnWriteArrayList
Multithreading	Seven threads and two executors	engine/*, db/*, app/*	Responsive concurrency	EDT, feed, intake, three workers, archive, control, database
Thread lifecycle	Controlled start and stop	engine/BasinEngine, engine/SensorFeed	Safe lifecycle	start, interrupt, join, getState, sleep
Runnable	Task versus execution	engine/SensorIntake, ResponseWorker	Separate task from thread	Runnable tasks scheduled on threads
Synchronization	Monitors	engine/EventBuffer, domain/ResourceAllocator, domain/BasinModel	Protect shared state	28 synchronized members; short critical sections
Monitors	Intrinsic lock and conditions	engine/EventBuffer	Condition signalling	Condition checked in a while loop under one monitor
Inter-thread communication	wait() and notifyAll()	engine/EventBuffer	Sleep rather than spin	Signal after every state change
Producer-consumer	Bounded buffer	engine/EventBuffer	Decouple rates and apply back-pressure	96-slot bounded buffer

Serialization	Session save	io/ SessionSnapshot, io/SnapshotStore	Persist a session	Serializable, serialVersionUID, ObjectOutputStream
Deserialization	Safe restore	io/ SnapshotStore	Recover a session	ObjectInputFilter allowlist and version/invariant checks
File handling	Local data management	io/AppPaths, io/SaveCatalog	Manage files	Files.createDirectories, listFiles, lastModified
JDBC and databases	Offline history	db/*	Durable, queryable history	Connection, PreparedStatement, ResultSet, SQLException, transactions

Table 4.1: Master mapping of object-oriented and Java concepts to the BasinWatch implementation.

4.2 Encapsulation

Definition. Encapsulation is the bundling of state with the methods that guard it, so that an object's internals cannot be changed arbitrarily from outside.

Where it appears. Encapsulation is most visible in the domain aggregate. BasinZone keeps its water level, rainfall, exposed population, levee state, and risk private, and exposes only the operations that make sense — apply a reading, pump water out, reinforce the levee, evacuate — plus a read-only snapshot. ResourceAllocator keeps its available and capacity maps private and exposes only tryReserve, release, and snapshot. The persistence DTO SessionSnapshot keeps its arrays private and validates them before exposing a domain snapshot.

Why it is needed. A basin model in which any code could set a zone's water level to an impossible value, or decrement a resource count without checking availability, would be a

model in which invariants are suggestions rather than guarantees. Encapsulation makes those invariants enforceable.

How it works. Fields are declared private. Mutating methods are package-private where only the domain needs them, so no code outside the package can alter a zone's water level directly. Where an object is constructed, the constructor validates its inputs; the immutable records validate in their compact constructors. Accessors return either primitives or immutable copies, never the mutable internal collection.

```
// domain/BasinZone.java (fields, condensed)
private final String id;
private final String name;
private double waterLevelMeters;
private double lastRainMillimeters;
private int exposedResidents;
private boolean leveeReinforced;
private long lastReadingTick = -1;
private RiskLevel risk;
```

Listing 4.1: Encapsulated zone state; only domain methods may change it.

Interaction with other classes. BasinModel is the only class that calls BasinZone's package-private mutators; the user interface never sees a BasinZone at all — it receives ZoneSnapshot records. This separation is what allows the UI to be updated from any thread safely.

Practical benefit. No external code can create an impossible zone or over-allocate a resource; the compiler and the constructor do the enforcing.

Syllabus link. Encapsulation, access modifiers, and the principle of information hiding.

4.3 Abstraction

Definition. Abstraction exposes the essential behaviour of a concept while hiding the details of its implementation.

Where it appears. The central abstraction is the BasinEvent interface. It declares four operations — the tick at which the event occurred, the zone it concerns, its category, and a human-readable description — and nothing else. Three concrete records implement it: SensorReading, SimulationPulse, and DispatchRequest. The persistence and engine layers also use small private functional interfaces (IoAction, IoSupplier, SqlAction, SqlSupplier) to abstract "an action that may fail".

```
// domain/BasinEvent.java
public interface BasinEvent {
    long tick();
    String zoneId();
    String category();
    String describe();
}
```

Listing 4.2: The event abstraction that the entire pipeline depends upon.

Why it is needed. The buffer, the workers, the journal, and the archive all handle events, but they must not be rewritten every time a new kind of event is added. The abstraction lets the pipeline be written once against BasinEvent and remain closed to modification when new event types appear.

How it works. The buffer stores BasinEvent references. When a worker processes an event, BasinModel dispatches on the concrete type to apply the correct transition. The journal records event.category() and event.describe() without knowing the event's class.

Interaction with other classes. EventBuffer depends only on BasinEvent. BasinEngine.process records the category and description generically. BasinModel.process is the single place that knows the concrete types.

Practical benefit. The producer, the buffer, and the persistence side are decoupled from the catalogue of event types, which is exactly the kind of stability that object-oriented design is meant to provide.

Syllabus link. Abstraction, interfaces, and programming to a contract.

4.4 Inheritance

Definition. Inheritance derives a class from a superclass so that it reuses and extends the superclass's behaviour.

Where it appears. Inheritance is used narrowly and only where the application genuinely adopts a framework contract. SensorFeed and ArchiveWorker extend Thread, because each owns a long-lived lifecycle and a blocking endpoint. MainFrame extends JFrame, BasinMapPanel and DatabaseHistoryPanel extend JPanel, MissionRenderer extends DefaultListCellRenderer, and HistoryTableModel extends AbstractTableModel, because Swing's components and models are designed to be extended.

Why it is needed. A feed that owns its own timed lifecycle and a component that must paint itself are both naturally expressed as subclasses. The alternative — composition with an internal Thread — is possible but would add indirection without adding clarity for these specific cases.

How it works. The subclass overrides the superclass hook. SensorFeed overrides run(); BasinMapPanel overrides paintComponent(Graphics); MissionRenderer overrides getListCellRendererComponent; HistoryTableModel implements the abstract table-model methods.

Interaction with other classes. The engine starts a SensorFeed as a Thread; the frame is displayed as a JFrame; the map panel is added to the frame as a JPanel.

Practical benefit. The application integrates with real platform facilities with very little code, and the overrides are exactly the hooks the frameworks intend developers to use.

Syllabus link. Inheritance, method overriding, and the is-a relationship. Note that the project deliberately avoids inheritance for the domain itself, where composition is more appropriate.

4.5 Polymorphism

Definition. Polymorphism allows one interface to stand for several concrete behaviours.

Where it appears. Polymorphism appears at three levels. First, event processing is polymorphic: BasinModel.process receives a BasinEvent and dispatches to the correct handler based on the concrete type. Second, mission effects are polymorphic over an enum: when a mission completes, a switch on the mission type decides whether to pump, reinforce, or evacuate. Third, presentation is polymorphic: risk categories map to colours through a switch, and history categories map to repository queries through a switch.

```
// domain/BasinModel.java (process, line 44)
if (event instanceof SensorReading reading) {
    applyReading(reading, messages);
} else if (event instanceof SimulationPulse pulse) {
    applyPulse(pulse, messages);
} else if (event instanceof DispatchRequest request) {
    applyDispatch(request, messages);
} else {
    throw new IllegalArgumentException("Unsupported basin event: " +
event.getClass().getName());
}
```

Listing 4.3: Polymorphic event dispatch, with an explicit rejection of unknown event types.

Why it is needed. Different events have genuinely different effects on the basin; different missions have genuinely different effects on a zone. Polymorphism keeps those differences in one place while allowing the surrounding pipeline to be uniform.

How it works. The runtime type of the event selects the handler. The switch on MissionType selects the effect. Because the enums are exhaustive, the compiler can help ensure every case is handled.

Interaction with other classes. The buffer, workers, and journal treat all events uniformly; only the model distinguishes them.

Practical benefit. New event types can be added with a single new handler and a single new branch, while the rest of the pipeline is untouched.

Syllabus link. Polymorphism, dynamic dispatch, and method overriding.

4.6 Composition

Definition. Composition is a strong whole-part relationship in which the whole owns its parts and controls their lifetime.

Where it appears. BasinModel is a composition root: it owns a LinkedHashMap of six zones, a LinkedHashMap of missions, an ArrayDeque of activity entries, and a ResourceAllocator. BasinEngine composes the EventBuffer, SimulationSettings, SnapshotStore, OperationalJournal, DatabaseService, and the worker threads. BasinSnapshot composes its lists of zone and mission snapshots.

Why it is needed. The parts of the model change together and must remain mutually consistent; making the aggregate own them keeps that consistency in one place. The engine's services likewise share a lifecycle and are started and stopped together.

How it works. The whole creates its parts in its constructor and holds them in final fields. The parts are not shared with unrelated code.

Interaction with other classes. The engine calls model.process(...) and model.snapshot(); it never reaches into the model's collections. The UI receives BasinSnapshot, which is a composed value.

Practical benefit. The model can guarantee that its zones and inventory always agree, because it is the only owner of both.

Syllabus link. Composition, aggregation, and the aggregate-root idea.

4.7 Association and Aggregation

Definition. Association is a general "uses" relationship; aggregation is a weaker whole-part relationship in which the part can outlive or be shared beyond the whole.

Where it appears. Association is visible between the engine and the database service, and between the UI and the engine: the UI holds a reference to the engine and registers listeners, but does not own the engine's threads. Aggregation appears in MissionType, whose requirements() returns a shared map from resource types to quantities, and in SimulationSettings, a single settings object shared by the feed and the engine.

Why it is needed. Not every relationship is ownership. The UI must observe the engine without owning it; the mission types must share a common resource vocabulary without each owning a private copy.

How it works. The UI registers a listener with the engine; the engine holds the settings and passes the same instance to the feed. The mission requirements map is created once and copied defensively on access.

Practical benefit. Collaboration without ownership keeps the design flexible and avoids accidental lifetime coupling.

Syllabus link. Association, aggregation, and the distinction between "has-a" and "owns-a".

4.8 Constructors

Definition. A constructor initialises an object and establishes its invariants.

Where it appears. Constructors are the primary guard rails of the domain. SensorReading and the other event records validate their inputs in compact constructors. ResourceAllocator validates that available stock lies within capacity. BasinModel has a public constructor that seeds the six zones and a private constructor used when restoring a saved model. ResponseMission has two constructors.

```
// domain/SensorReading.java (compact constructor)
public SensorReading {
    if (tick < 0 || !validIdentifier(stationId) || !validIdentifier(zoneId)
            || !Double.isFinite(rainfallMillimeters) || rainfallMillimeters < 0
            || rainfallMillimeters > 500
            || !Double.isFinite(observedRiseCentimeters) || observedRiseCentimeters < 0
```

```
            || observedRiseCentimeters > 200) {
        throw new IllegalArgumentException("Invalid sensor reading.");
    }
}
```

Listing 4.4: A compact record constructor rejects impossible readings before the object exists.

Why it is needed. If an object can be constructed in an invalid state, every later method must defend against that state. Validating at construction removes an entire class of defensive checks.

How it works. The compact constructor runs before the record's fields are assigned, so an invalid record can never come into existence. The `BasinModel` constructors ensure the zone set is complete.

Practical benefit. The rest of the code can assume that any existing `SensorReading` or `BasinZone` is valid.

Syllabus link. Constructors, constructor overloading, and object initialisation.

4.9 Access Modifiers

Definition. Access modifiers control the visibility of classes, fields, and methods.

Where it appears. Throughout the codebase. Public API is limited to what callers need: the engine's lifecycle methods, the domain's snapshots and events, and the persistence entry points. Internal helpers are private. Mutators that only the domain needs are package-private. Exactly one member is `protected` — `BasinMapPanel.paintComponent` — because Swing requires subclasses to override it.

Why it is needed. A wide public surface invites misuse. Restricting visibility makes the intended use obvious and prevents accidental coupling.

How it works. `private` for implementation details, package-private for domain-internal collaboration, `public` for genuine API, and `protected` only where a framework requires an overridable hook.

Practical benefit. The compiler enforces the design's intent about who may change what.

Syllabus link. Access modifiers and encapsulation.

4.10 Interfaces and the Deliberate Absence of Abstract Classes

Definition. An interface declares a contract without implementation; an abstract class provides a partial implementation that subclasses complete.

Where it appears. The domain's BasinEvent is the project's central interface. Four small private functional interfaces abstract fallible actions in the engine and database layers. The application contains **no abstract classes**. This is a deliberate decision: the only inheritance in the project is the narrow, purposeful adoption of framework contracts, and none of those required a shared partial implementation. Introducing an abstract class would have added a layer without serving the domain.

Why it is needed. The interfaces decouple the pipeline from concrete types. The absence of abstract classes keeps the inheritance graph shallow and easy to reason about.

Practical benefit. Readers can hold the entire type hierarchy in their heads; there is no speculative abstraction to navigate.

Syllabus link. Interfaces, abstract classes, and the judgement to use the right tool.

4.11 Enums

Definition. An enum defines a fixed set of named constants, which may carry fields and behaviour.

Where it appears. Five enums give the domain a closed vocabulary: RiskLevel (with a label and a numeric rank), MissionType (with a label, a duration in ticks, a water effect, and a resource requirement map), MissionStatus, ResourceType, and DatabaseHistoryType.

```
// domain/MissionType.java
PUMP_DEPLOYMENT("Deploy portable pump", 8, 0.62,
        Map.of(ResourceType.PUMP, 1, ResourceType.FIELD_CREW, 1)),
LEVEE_REINFORCEMENT("Reinforce levee", 10, 0.0,
        Map.of(ResourceType.LEVEE_CREW, 1, ResourceType.SUPPLY_TRUCK, 1)),
EVACUATION("Evacuate exposed residents", 6, 0.0,
        Map.of(ResourceType.EVAC_VEHICLE, 1, ResourceType.FIELD_CREW, 1));
```

Listing 4.5: Mission types carry their own duration, effect, and resource requirements.

Why it is needed. A risk level or mission type expressed as a string would permit typographical errors and invalid values. An enum makes the set closed and the values discoverable.

How it works. Enum constructors attach data to each constant. EnumMap uses the enum ordinals for compact, fast storage. switch over enums is exhaustive and readable.

Practical benefit. Invalid categories become impossible, and the code reads like the domain.

Syllabus link. Enumerations and their use in modelling fixed domains.

4.12 Immutable Objects

Definition. An immutable object cannot change after construction, so it can be shared freely between threads without locking.

Where it appears. The project defines fourteen record types, including the events, the snapshots (ZoneSnapshot, MissionSnapshot, BasinSnapshot), ActivityEntry, DatabaseHistoryEntry, ThreadDiagnostic, EngineDiagnostics, and AppPaths. Each record's components are final, and the snapshot records copy their collections defensively.

```
// domain/BasinSnapshot.java (compact constructor)
public BasinSnapshot {
    zones = List.copyOf(zones);
    availableResources = Map.copyOf(availableResources);
    missions = List.copyOf(missions);
    activity = List.copyOf(activity);
}
```

Listing 4.6: A snapshot copies its collections so that no later mutation can alter it.

Why it is needed. This is the mechanism that makes the concurrency design safe. Background workers mutate the live model under a lock; the interface reads an immutable snapshot without a lock. Because the snapshot cannot change, there is no possibility of the interface observing a half-applied update.

How it works. The model builds a snapshot from copies of its state. The snapshot's collections are unmodifiable. The interface may hold the snapshot for as long as it likes.

Practical benefit. Thread-safe hand-off with no locks on the reading side — one of the most valuable outcomes of the entire design.

Syllabus link. Immutability, final, and defensive copying.

4.13 Event-Based Design

Definition. Event-based design structures a system around the production, transport, and consumption of events.

Where it appears. The whole pipeline is event-based. The sensor feed produces events; the intake task validates them; the buffer transports them; the workers consume them. The engine also exposes three listener lists (`snapshotListeners`, `statusListeners`, `databaseStatusListeners`) so that the interface can subscribe to state changes without polling.

Why it is needed. The domain is naturally event-shaped: things happen over time, and the system must react to each. Event-based design lets production and consumption proceed independently.

How it works. Events are immutable values carried by the buffer. Listeners are held in `CopyOnWriteArrayList` instances, which are safe to iterate while other threads modify them. The UI subscribes and receives updates on the EDT.

Practical benefit. Loose coupling between producer, consumer, and presentation; the UI never needs to ask "has anything changed?".

Syllabus link. Event-driven programming and the observer pattern.

4.14 Exception Handling

Definition. Exception handling is the language's mechanism for detecting and responding to abnormal conditions.

Where it appears. Every boundary in the system. Domain constructors throw `IllegalArgumentException`; invalid state transitions throw `IllegalStateException`; file operations throw `IOException`; database operations throw `SQLException`; the intake task catches malformed records and reports them without stopping.

Why it is needed. Malformed telemetry, corrupt saves, resource shortages, and database outages are different problems that must be distinguishable. Treating them all as "something went wrong" would make the system undiagnosable and would risk presenting failure as success.

How it works. Each boundary validates and throws a specific exception. Callers either handle the exception meaningfully or let it propagate with context. Try-with-resources guarantees that streams and connections are closed. Interrupted threads preserve their interrupt status so that the shutdown path behaves correctly.

Practical benefit. The operator sees an accurate account of what happened, and unrelated work continues when one part fails.

Syllabus link. Exception handling, checked versus unchecked exceptions, and resource management.

4.15 OOP Design Benefits

The benefits of this design are concrete and observable. Encapsulation means the domain cannot be driven into an invalid state from outside. Abstraction means the pipeline is stable against new event types. Narrow inheritance means the type graph is shallow and legible. Polymorphism localises the differences between events, missions, and risks. Composition gives the model a single owner for state that must remain consistent. Enums remove an entire class of stringly-typed errors. Immutability is what makes concurrent reads safe. Event-based design keeps producer, consumer, and interface decoupled. Disciplined exception handling keeps failures visible and local. Together these choices produce a program that is not merely written in an object-oriented language but is designed in an object-oriented way — which is the point of the exercise.

CHAPTER 5: JAVA I/O AND PERSISTENCE

5.1 Java I/O Overview

Java's I/O library is organised around two parallel hierarchies: **byte streams**, which move raw bytes and are the correct choice for binary data, and **character streams**, which move text and handle character encoding. Both hierarchies are built from the same idea: a stream is a source or sink of data, and streams can be wrapped around one another so that each layer adds one concern. BasinWatch uses both hierarchies, and it chooses between them deliberately: typed measurements and session snapshots are binary, so they use byte streams; the operational journal and the situation report are text, so they use character streams.

5.2 Byte Streams

Byte streams are represented by InputStream and OutputStream. The project's binary archive and its session snapshots are built on these base classes. SensorArchive writes through a DataOutputStream layered over a BufferedOutputStream layered over a file output stream, so that the file ultimately receives bytes. The snapshot store reads and writes bytes through byte-array streams before committing them to a file.

5.3 Character Streams

Character streams are represented by Reader and Writer. The operational journal and the report writer both use a BufferedWriter created from Files.newBufferedWriter with an explicit UTF-8 charset, so that timestamps and human-readable messages are encoded consistently regardless of the platform default. The sensor pipeline's internal transport is also character-based: the feed writes to a PipedWriter and the intake task reads through a PipedReader wrapped in a BufferedReader.

5.4 Buffered Streams

Buffering reduces the number of low-level operating-system calls by batching small reads and writes. The archive uses a BufferedOutputStream, the journal uses a BufferedWriter, and the intake task reads through a BufferedReader. This matters because the application writes many small items — one journal line per event, one typed record per reading — and without buffering each item would incur its own file operation.

5.5 Data Streams

Data streams read and write Java's primitive types in a defined, portable order. SensorArchive uses DataOutputStream to write each measurement as a long tick, two UTF strings, and two double values. The advantage over plain text is that types are preserved exactly: a water level is stored as an eight-byte double, not as a decimal string that would need parsing and could lose precision or depend on a locale.

```
// io/SensorArchive.java (append, line 40)
public synchronized void append(SensorReading reading) throws IOException {
    output.writeLong(reading.tick());
    output.writeUTF(reading.stationId());
    output.writeUTF(reading.zoneId());
    output.writeDouble(reading.rainfallMillimeters());
    output.writeDouble(reading.observedRiseCentimeters());
}
```

Listing 5.1: Typed binary records written in a fixed order.

5.6 Filter and Wrapper Streams

The filter streams are the wrapping streams that add behaviour: buffering, typed encoding, or object serialization. The project relies on this pattern repeatedly. The archive is a data stream wrapped around a buffered stream wrapped around a file stream. The journal is a buffered writer around a file. The snapshot store is an object stream around a byte-array stream around a file. Each layer adds exactly one concern, and the domain never needs to know which concrete device lies at the bottom.

Artefact	Stream chain	Data
Operational journal	BufferedWriter → file	UTF-8 text lines
Sensor archive	DataOutputStream → BufferedOutputStream → file	Typed binary records
Session save	ObjectOutputStream → ByteArrayOutputStream → file	Serialized snapshot
Session load	file → BufferedInputStream → ByteArrayInputStream → ObjectInputStream	Deserialized snapshot

Situation report	BufferedWriter → file	UTF-8 text

Table 5.1: Persistence artefacts and their stream chains.

5.7 Byte-Array Streams

Byte-array streams operate on in-memory arrays rather than files. The snapshot store uses ByteArrayOutputStream to build a complete serialized payload in memory and ByteArrayInputStream to read it back. This matters because it allows the store to measure and validate the entire payload *before* it touches the target file, so a save either succeeds completely or leaves the previous save untouched. A subtlety worth recording, and one the syllabus emphasises, is that a byte-array input stream does not grow its source array: it reads only the bytes it was given.

5.8 File and File Metadata

File and Path describe locations and metadata rather than contents. The AppPaths record resolves the per-user data root, creates the four data directories, and exposes the database path. The SaveCatalog lists session saves by examining file metadata — it filters names ending in .bws and sorts by last-modified time — without opening any file's contents.

```
// io/AppPaths.java (create, line 36)
Path absolute = root.toAbsolutePath().normalize();
Path logs = absolute.resolve("data").resolve("logs");
Path archive = absolute.resolve("data").resolve("archive");
Path saves = absolute.resolve("data").resolve("saves");
Path reports = absolute.resolve("data").resolve("reports");
Path database = absolute.resolve("data").resolve("basinwatch.db");
Files.createDirectories(logs);
Files.createDirectories(archive);
Files.createDirectories(saves);
Files.createDirectories(reports);
```

Listing 5.2: Directory and path resolution using `Path` and `Files`.

On Windows the default data root is %APPDATA%\BasinWatch; on other platforms it is ~/.basinwatch. A --data-dir argument overrides the location.

5.9 Serialization

Serialization converts an object graph into a byte stream. The project serializes a SessionSnapshot, a Serializable class that carries the simulation tick, the six zones, the resource counts, the missions, and recent activity. The class declares an explicit

serialVersionUID and a format version, and one field is marked transient so that it is never written to disk.

5.10 Deserialization

Deserialization reconstructs the object graph. The snapshot store deserializes carefully: it checks the file size, reads at most the permitted number of bytes, and installs an ObjectInputFilter before reading. The filter allows only primitives, strings, and the application's own snapshot classes, and rejects graphs that are too deep or too large. After reading, the store validates the format version and re-checks the domain invariants before handing a snapshot to the model.

```
// io/SnapshotStore.java (filter, line 100)
private static ObjectInputFilter.Status filter(ObjectInputFilter.FilterInfo info) {
   if (info.depth() > 12 || info.references() > 2_000 || info.streamBytes() > MAX_SAVE_BYTES)
{
      return ObjectInputFilter.Status.REJECTED;
   }
   Class<?> type = info.serialClass();
   if (type == null) {
      return ObjectInputFilter.Status.UNDECIDED;
   }
   if (type.isPrimitive() || type == String.class
         || type == SessionSnapshot.class || type == SavedZone.class
         || type == SavedMission.class || type == SavedActivity.class) {
      return ObjectInputFilter.Status.ALLOWED;
   }
   // arrays of allowed components ... otherwise REJECTED
   return ObjectInputFilter.Status.REJECTED;
}
```

Listing 5.3: An allowlist filter that confines deserialization to the application's own snapshot classes.

5.11 `transient` and Runtime-Only State

Some state must never be serialized: locks, live threads, open stream handles, and references to Swing components. SessionSnapshot marks its sourcePathHint field transient, and, more importantly, the snapshot deliberately contains only *data* — never a live ResourceAllocator monitor, never a thread, never a Swing object. When a session is restored, the model reconstructs fresh runtime coordination primitives. This is the practical meaning of the syllabus point that runtime-only state belongs to the live process, not to the saved file.

5.12 Session Snapshot

The SessionSnapshot is a versioned data-transfer object. On save it is built from an immutable BasinSnapshot; on load it is converted back into a BasinSnapshot and then validated by BasinModel.restore. Because the snapshot is a flat DTO rather than the live aggregate, the save file is compact, stable, and independent of the runtime structure.

5.13 Sensor Archive

The sensor archive is a versioned binary file. Its first two integers are a magic marker (0x42574152) and a format version (1); when an existing archive is opened, the constructor reads and checks these before appending, so a file that is not a BasinWatch archive — or that was written by an incompatible version — is rejected immediately.

5.14 Operational Journal

The operational journal is an append-only UTF-8 text file. Every event message is written with a timestamp and category. To avoid an operating-system call per line, the journal buffers writes and flushes only every twelfth line, or immediately when the category contains WARNING, COMPLETE, or ALLOCATED — the categories an operator is most likely to want preserved promptly.

```
// io/OperationalJournal.java (append, line 22)
public synchronized void append(ActivityEntry entry) throws IOException {
    writer.write(DateTimeFormatter.ISO_INSTANT.format(entry.at()));
    writer.write(" | ");
    writer.write(entry.category().replace('\n', ' '));
    writer.write(" | ");
    writer.write(entry.message().replace('\n', ' '));
    writer.newLine();
    pendingLines++;
    if (pendingLines >= 12 || entry.category().contains("WARNING")
            || entry.category().contains("COMPLETE") || entry.category().contains("ALLOCATED"))
{
        writer.flush();
        pendingLines = 0;
    }
}
```

Listing 5.4: Buffered journal writes with category-sensitive flushing.

5.15 Report Generation

The report writer produces a human-readable situation report in UTF-8 text. Each report has four sections — zone conditions, available resources, response operations, and recent activity — and a timestamped filename that includes a short random suffix so that successive reports never collide.

5.16 Save and Restore Workflow

Figure 5.1 shows the complete save and restore workflow. Saving pauses the feed if it is live, waits for accepted events to drain, builds and validates the snapshot, encodes it to a bounded byte array, writes it to a temporary file, and then moves that file into place with an atomic move where the file system supports it. Restoring checks the file size, applies the input filter, validates the format version and the domain invariants, and rebuilds fresh runtime primitives.

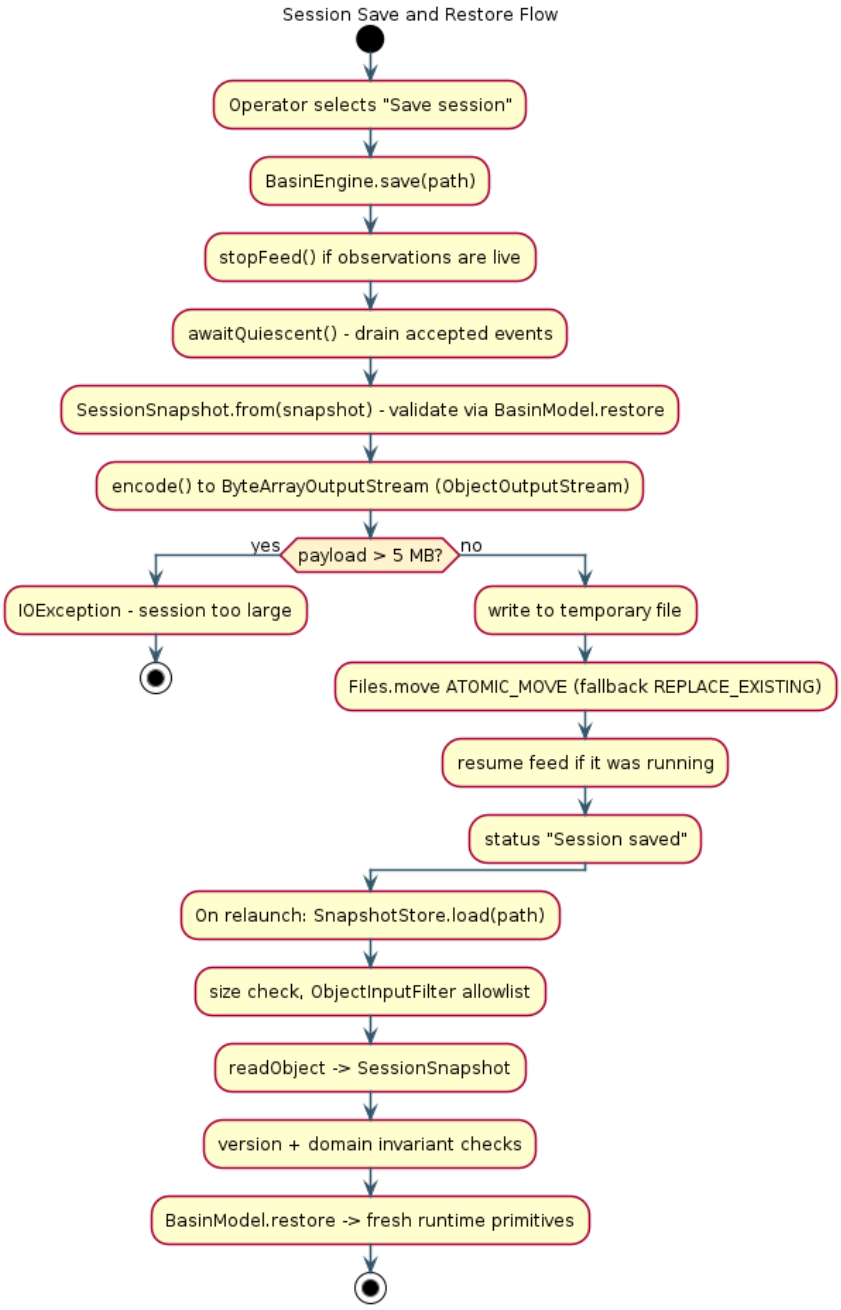


Figure 5.1: Session save and restore flow.

The atomic move is the reason the save is safe against interruption: a partially written temporary file never replaces a valid save. If the move must fall back to a non-atomic replacement on a file system that does not support atomic moves, the code does so explicitly rather than silently, and it deletes the temporary file if the move did not happen.

CHAPTER 6: MULTITHREADING AND CONCURRENCY

6.1 Need for Concurrency

The simulation is concurrent by nature. Observations must be produced on their own schedule, independently of how quickly the operator acts and of how quickly earlier observations are processed. The interface must remain responsive while that happens. Missions progress independently of any single event. If all of this were done on one thread, the interface would stall whenever the simulation worked, and the very behaviour the project exists to demonstrate — sustained coordination under change — would be impossible.

6.2 Process Versus Thread

A process has its own address space and resources; a thread shares the address space of its process with its siblings. BasinWatch is a single process. Within it, several threads share one heap, which is precisely why the shared model must be guarded: threads can see each other's data, so correctness depends on synchronisation rather than on isolation.

6.3 Thread Lifecycle

Java's Thread.State enumeration defines the states a thread can occupy: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. Two clarifications matter for correctness. First, Java does **not** define a separate RUNNING state; a thread that is executing or ready to execute is reported as RUNNABLE. Second, a thread that calls wait() enters WAITING and **releases** its monitor, whereas a thread that calls Thread.sleep() enters TIMED_WAITING and **does not** release its monitor. These distinctions are not academic; they determine whether other threads can make progress. Figure 6.1 shows the states as BasinWatch uses them.

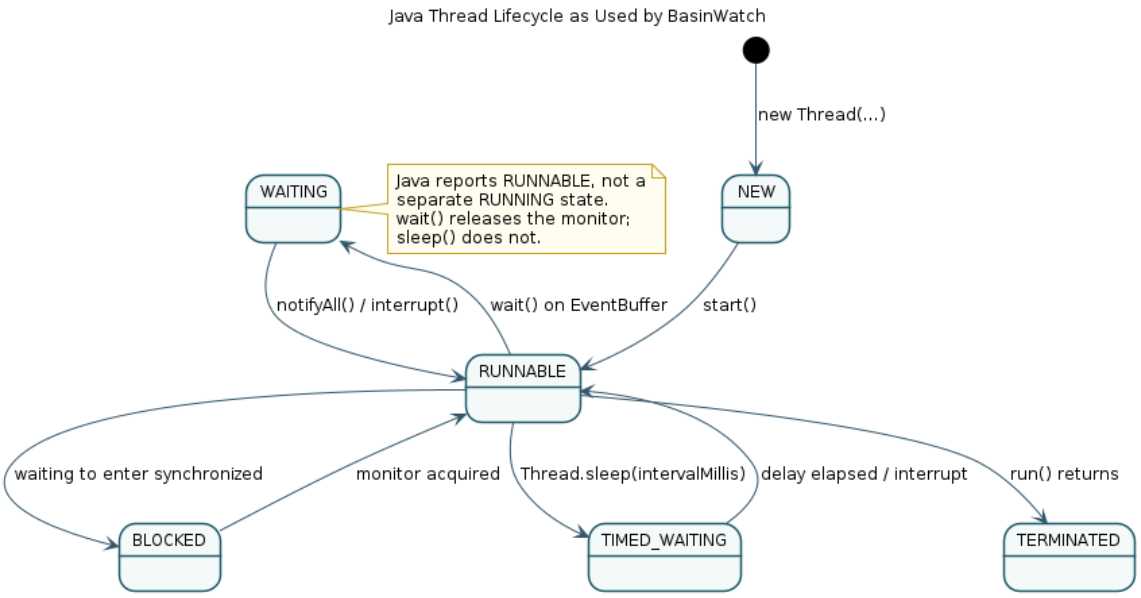


Figure 6.1: Java thread lifecycle as used by BasinWatch.

6.4 Thread

Thread represents an execution path. The project uses two subclasses — SensorFeed and ArchiveWorker — because each owns a distinct lifecycle and a blocking endpoint. Both are given explicit names (basin-sensor-feed, basin-sensor-archive) so that diagnostics can report them truthfully.

```
// engine/SensorFeed.java (run, line 33)
public void run() {
    try {
        while (running && !isInterrupted()) {
            long tick = tickSequence.incrementAndGet();
            for (int index = 0; index < zoneIds.size() && running; index++) {
                double rainfall = (2.0 + random.nextDouble() * 14.0) * settings.stormIntensity();
                double rise = (0.3 + random.nextDouble() * 1.6) * settings.stormIntensity();
                SensorReading reading = new SensorReading(tick, "G-" + zoneIds.get(index),
                        zoneIds.get(index), rainfall, rise);
                destination.write(reading.encode());
                destination.write('\n');
            }
            destination.write("PULSE|" + tick + '\n');
            destination.flush();
            Thread.sleep(settings.intervalMillis());
        }
    } catch (InterruptedException ex) {
        interrupt();
    } catch (IOException ex) {
        if (running) {
            failureReporter.accept("Sensor channel stopped unexpectedly: " + ex.getMessage());
        }
    } finally {
        try {
            destination.close();
```

```
        } catch (IOException ex) {
            if (running) {
                failureReporter.accept("Could not close sensor channel: " + ex.getMessage());
            }
        }
    }
}
```

Listing 6.1: The sensor feed writes one interval of observations, flushes, and then sleeps until the next interval.

6.5 Runnable

Runnable separates a task from the thread that executes it. The intake task and the response workers implement Runnable rather than extending Thread, because they are jobs that can be scheduled by worker threads. This is the recommended division: inherit Thread only when the thread itself has a lifecycle to own; implement Runnable when the work is a task.

```
// engine/SensorIntake.java (run, line 26)
public void run() {
    try (source) {
        String line;
        while ((line = source.readLine()) != null) {
            BasinEvent event;
            try {
                if (line.startsWith("PULSE|")) {
                    event = new SimulationPulse(Long.parseLong(line.substring(6)));
                } else {
                    event = SensorReading.parse(line);
                }
            } catch (IllegalArgumentException ex) {
                warningReporter.accept("Discarded malformed sensor record: " + ex.getMessage());
                continue;
            }
            if (!destination.put(event)) {
                return;
            }
            acceptedReporter.accept(event.category());
        }
    } catch (IOException ex) {
        warningReporter.accept("Sensor intake ended with an I/O error: " + ex.getMessage());
    }
}
```

Listing 6.2: The intake task reads complete lines, validates each record, and enqueues accepted events.

6.6 Sensor Feed

The sensor feed is the producer. It increments a shared tick counter, emits one reading per zone, writes a pulse to mark the interval, flushes the pipe, and sleeps until the next interval. Its loop condition checks both a running flag and the interrupt status, so shutdown is cooperative rather than abrupt.

6.7 Sensor Intake

The intake task is the bridge between the character-pipe transport and the event buffer. It reads complete lines, distinguishes pulses from readings, parses and validates each record, and enqueues accepted events. A malformed record is logged and discarded; it never terminates the intake task, which is what allows the simulation to survive noisy input.

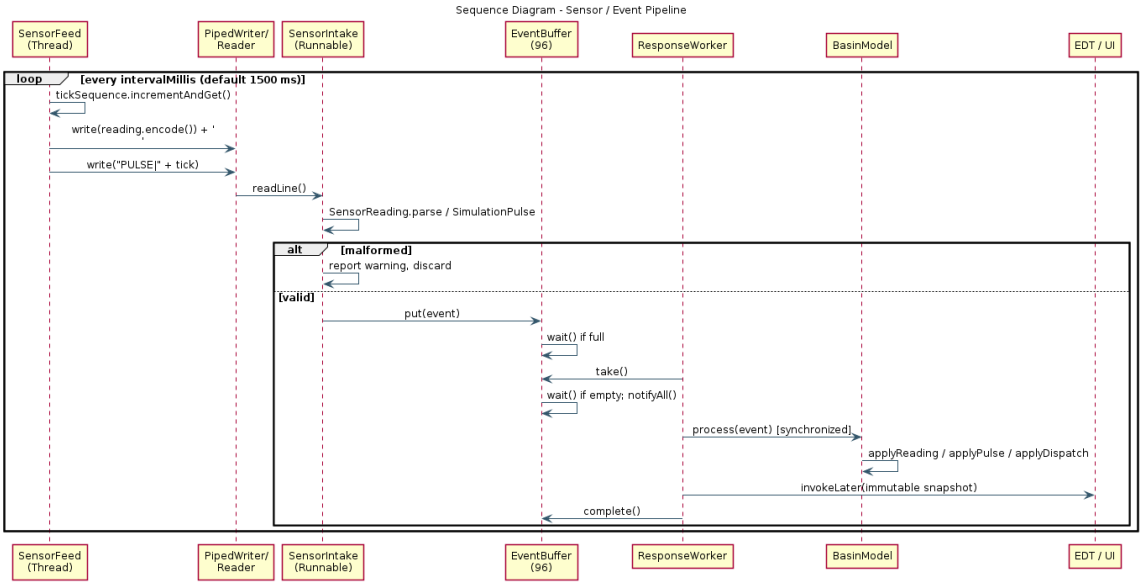


Figure 6.5: Sequence diagram of the sensor and event pipeline.

6.8 Event Buffer

The event buffer is a bounded, monitor-protected queue of capacity ninety-six. It is the point at which production and consumption meet, and it is where the project's most important synchronisation lives.

Operation	Condition	Behaviour
put(event)	Buffer full	Waits until space is available or the buffer closes

offerAll(batch)	Batch larger than free space	Rejects the whole batch without partial insertion
take()	Buffer empty	Waits until an event arrives or the buffer closes
complete()	No event in flight	Throws; otherwise decrements the in-flight count
close()	Any	Sets closed and wakes all waiters

Table 6.1: Event buffer operations and their conditions.

6.9 Producer-Consumer Pattern

The producer-consumer pattern decouples the rate of production from the rate of consumption through a buffer. In BasinWatch, the intake task produces and the response workers consume. The buffer is bounded, so a fast producer cannot exhaust memory; instead it experiences back-pressure and waits. The pattern is what allows the two sides to run at their own speeds while remaining correct.

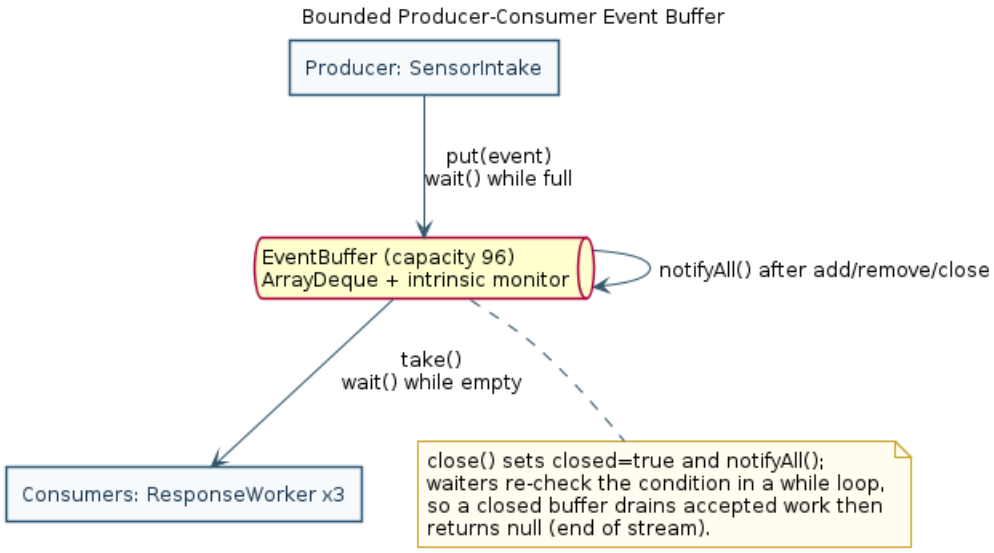


Figure 6.2: The bounded producer-consumer event buffer.

6.10 `wait()`

wait() suspends the calling thread until another thread signals it, and it **releases the object's monitor** while the thread waits, so other threads can enter the synchronized region. In BasinWatch, a consumer calls wait() while the buffer is empty and a producer calls wait() while the buffer is full.

6.11 `notifyAll()`

notifyAll() wakes every thread waiting on the monitor. The project uses notifyAll() rather than notify() because both producers and consumers may be waiting, and it is impossible to know which single thread should be woken. Every state change — adding an event, removing an event, completing an event, or closing the buffer — is followed by notifyAll().

```
// engine/EventBuffer.java (put, line 23)
public synchronized boolean put(BasinEvent event) throws InterruptedException {
    if (event == null) {
        throw new IllegalArgumentException("Event is required.");
    }
    while (!closed && events.size() == capacity) {
        waitingProducers++;
        try {
            wait();
        } finally {
            waitingProducers--;
        }
    }
    if (closed) {
        return false;
    }
    events.addLast(event);
    notifyAll();
    return true;
}
```

Listing 6.3: A producer waits in a `while` loop while the buffer is full, then signals on insertion.

6.12 Synchronization

Synchronisation prevents two threads from executing a critical section simultaneously. The project uses twenty-eight synchronized members. The critical sections are deliberately short: they contain validation and state arithmetic, never I/O and never a wait on another monitor.

6.13 Monitor Locks

Every Java object has an intrinsic monitor, entered by a synchronized block or method. The event buffer's methods are synchronized on the buffer itself, so put, take, complete, and close are mutually exclusive and their condition checks are atomic with respect to one another. The buffer's condition checks occur in while loops rather than if statements, because a woken thread must re-check the condition: between the wake-up and the reacquisition of the monitor, another thread may have changed the state.

6.14 Resource Reservation

Response missions require several resource types at once. A pump deployment needs a pump and a field crew; a levee reinforcement needs a levee crew and a supply truck; an evacuation needs an evacuation vehicle and a field crew. The allocator must reserve all of a mission's requirements or none of them. Table 6.2 lists the standard resource catalogue, and Table 6.3 lists each mission type with its duration, effect, and requirements.

Resource type	Standard stock
Portable pump	3
Levee crew	3
Supply truck	2
Evacuation vehicle	2
Field crew	4

Table 6.2: Response resource catalogue and standard stock.

Mission type	Duration (ticks)	Water effect (m)	Requirements
Pump deployment	8	0.62	1 pump, 1 field crew
Levee reinforcement	10	0.0	1 levee crew, 1 supply truck
Evacuation	6	0.0	1 evacuation vehicle, 1 field crew

Table 6.3: Mission types, durations, effects, and resource requirements.

```
// domain/ResourceAllocator.java (tryReserve, line 36)
public synchronized boolean tryReserve(Map<ResourceType, Integer> request) {
    validateRequest(request);
    for (Map.Entry<ResourceType, Integer> entry : request.entrySet()) {
        if (available.get(entry.getKey()) < entry.getValue()) {
            return false;
        }
    }
    request.forEach((type, amount) -> available.put(type, available.get(type) - amount));
    return true;
}
```

Listing 6.4: The check and the decrement occur in one synchronized method, so reservation is all-or-nothing.

6.15 Atomic Resource Allocation

The reservation is atomic because the availability check and the decrement happen inside a single synchronized method. Two dispatches running at the same instant cannot both observe the same final pump as available: the first to acquire the monitor decrements it, and the second finds it unavailable. Figure 6.3 shows the flow, and the test suite demonstrates it with twenty-four simultaneous contenders, of which exactly one succeeds.

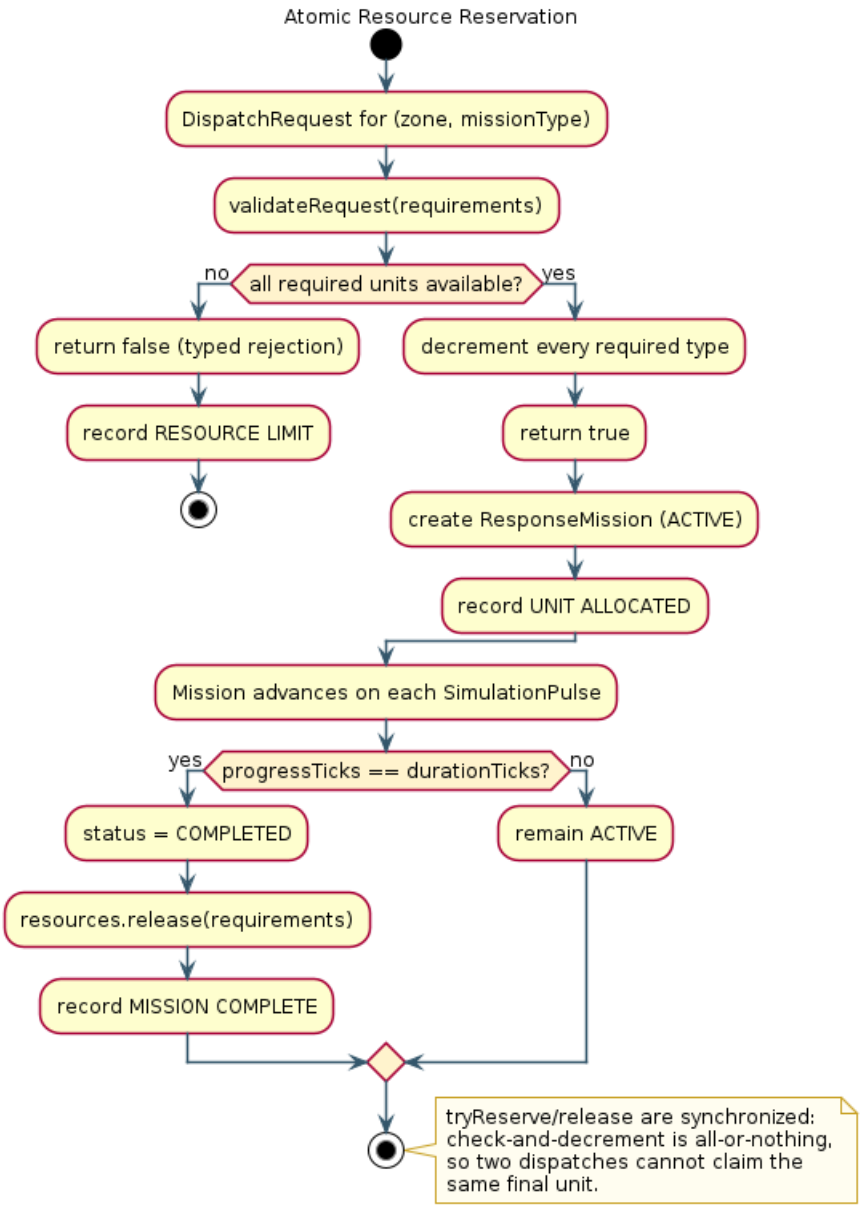


Figure 6.3: Atomic resource reservation and release.

6.16 Concurrent Missions

Several missions can be active at once, each with its own progress. Because missions advance deterministically on simulation pulses rather than on worker sleep timing, their progress does not depend on which worker happened to process which event. On completion, a mission releases exactly the resources it reserved.

6.17 Deadlock Risk

A deadlock occurs when two or more threads each hold a resource the others need, producing a circular wait. The classic hazard in a system like this would be an allocator that acquired several per-resource locks in different orders for different missions: one mission might hold the pump lock and wait for the crew lock while another holds the crew lock and waits for the pump lock. BasinWatch avoids this hazard by construction.

6.18 Deadlock Prevention

Prevention rests on three rules. First, there is a **single** inventory monitor: all reservations go through one synchronized allocator method rather than through several per-resource locks, so there is no lock order to get wrong. Second, no lock is held during I/O, sleep, callbacks, or a wait on a different monitor, so a thread never holds one resource while blocking on another. Third, where the model and the inventory must change together, the model owns the documented lock order. The result is that the normal operation of the application has no circular-wait precondition. It is worth stating plainly that synchronisation alone does not prevent all deadlocks; what prevents this particular deadlock is the single-monitor discipline and the refusal to block while holding a lock.

6.19 Thread Interruption

Interruption is the cooperative mechanism by which one thread asks another to stop. The project's workers and the feed check their interrupt status and, when interrupted, preserve the status (`Thread.currentThread().interrupt()`) and return. Preserving the interrupt status matters because a caller further up the stack may still need to know that an interruption occurred.

6.20 Graceful Shutdown

Shutdown is explicit and bounded. The engine stops the feed, closes the event buffer (which wakes every waiter), joins each response worker with a timeout, drains and finishes the archive worker, closes the database service, writes an automatic last-session.bws, and closes the journal. If any join times out or any I/O fails, the engine reports the failure rather than pretending the shutdown succeeded.

```
// engine/BasinEngine.java (closeEngine, condensed from line 340)
IOException failure = null;
try { stopFeed(); } catch (IOException ex) { failure = ex; }
eventBuffer.close();
for (Thread worker : List.copyOf(responseThreads)) { join(worker, "response worker"); }
if (failure != null || workersAlive()) {
    throw failure != null ? failure
        : new IOException("Background work is still active; operational files remain open.");
}
if (archiveWorker != null && archiveWorker.isAlive()) {
    archiveWorker.finish();
    join(archiveWorker, "sensor archive");
}
databaseService.close();
snapshotStore.save(paths.saves().resolve("last-session.bws"), model.snapshot());
journal.close();
closed = true;
```

Listing 6.5: Shutdown drains and closes each service in order, reporting rather than hiding failures.

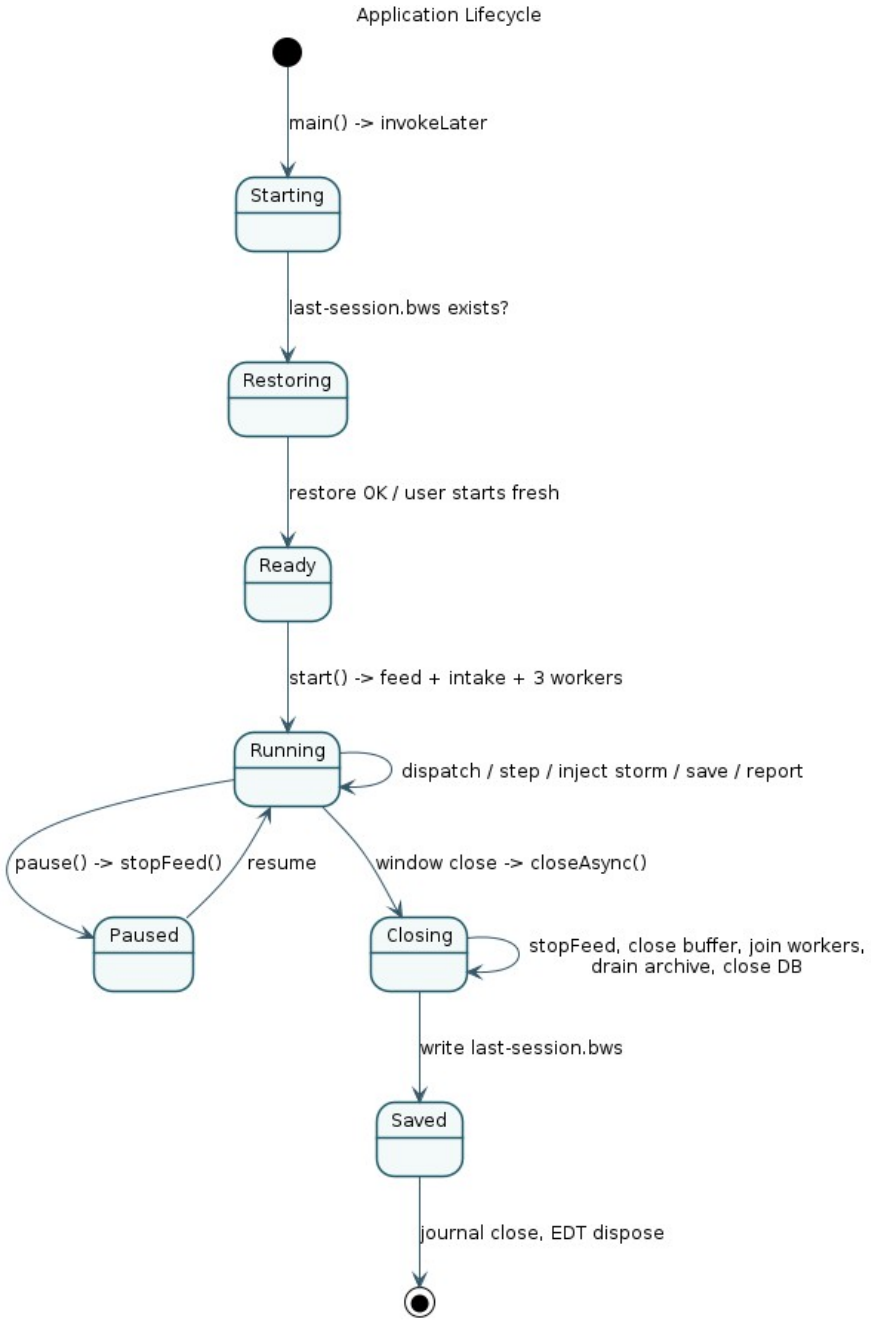


Figure 6.4: Application lifecycle from startup to clean close.

6.21 Thread Priority and Experimental Diagnostics

Java thread priority is a scheduling **hint**, not a guarantee. The platform is free to ignore it, and it must never be used as a correctness or ordering mechanism. BasinWatch respects this: operational ordering comes from explicit severity and timestamp data, not from priority. The one place priority appears is an optional experiment in the systems inspector, which can lower the archive worker's priority to demonstrate that non-urgent background work can be

de-emphasised — while the application's correctness remains independent of the setting. The inspector also reports each thread's name, state, and priority, so the operator can see the truth of the scheduling situation rather than assume it.

6.22 Concurrency Safety

Concurrency safety in BasinWatch rests on four pillars: a bounded buffer whose condition checks are protected by one monitor and re-checked in loops; a single short critical section for resource reservation; immutable snapshots for every hand-off to the interface; and cooperative interruption with bounded joins for shutdown. Together these make the application safe under concurrent operation, and the test suite exercises each pillar directly.

CHAPTER 7: DATABASE AND JDBC

7.1 Need for Database History

The file journal and binary archive preserve what happened, but they are not convenient to query. An operator who wants to see the twenty most recent readings in a particular zone, or to list every completed mission, would otherwise have to read and parse files. A relational database provides indexed, structured queries for exactly this purpose. BasinWatch therefore adds an **optional** embedded database as a supplementary history store. It is deliberately supplementary: the archive, journal, reports, and session snapshots remain the primary persistence mechanisms, and the database is never a dependency for the simulation.

7.2 SQLite

SQLite is an embedded, file-based relational database. There is no server process to install, start, or connect to; the database is a single file, and the JDBC driver links the database engine directly into the application. This makes SQLite an excellent fit for an offline desktop application: it provides real SQL and real transactions without any operational infrastructure. In BasinWatch the database file is `data/basinwatch.db`, placed beside the application's other data folders.

7.3 JDBC Architecture

JDBC is Java's standard API for relational databases. An application uses the same `Connection`, `PreparedStatement`, and `ResultSet` types regardless of the underlying database, while a driver translates those standard calls into the database's own protocol. BasinWatch uses the Xerial SQLite JDBC driver, declared in `pom.xml` and bundled into the shaded JAR so that the run command works without a separate classpath. Figure 7.1 shows the layer.

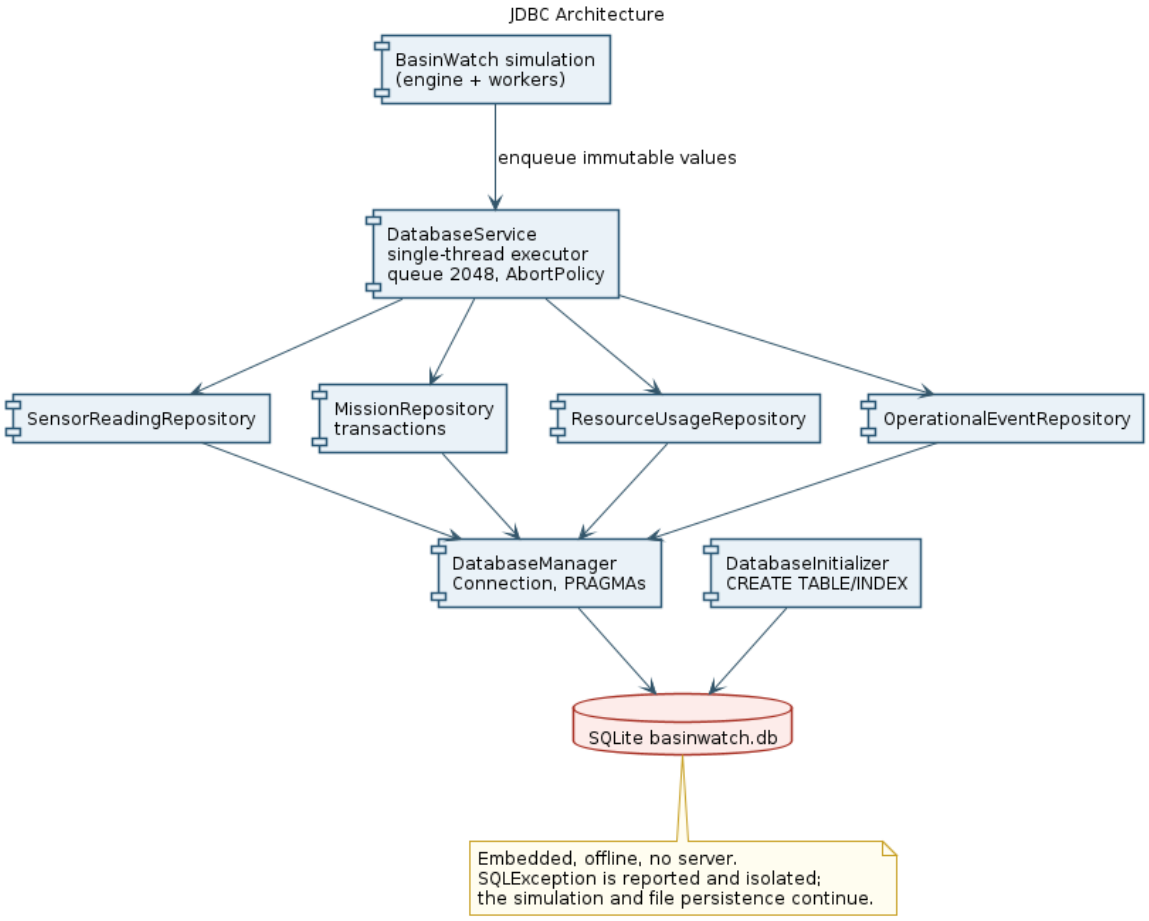


Figure 7.1: JDBC architecture of the BasinWatch database layer.

JDBC concept	BasinWatch implementation
JDBC driver	Xerial sqlite-jdbc declared in pom.xml
Connection	DatabaseManager.openConnection() opens and configures a connection
PreparedStatement	Repositories bind values with setString, setDouble, setInt
ResultSet	Repository findRecent methods read rows into immutable entries
SQLException	DatabaseService reports failures; the simulation continues
Transaction	MissionRepository commits a mission with its resources and event together

Table 7.1: JDBC concepts and their BasinWatch implementations.

7.4 Connection

Connections are opened per repository operation and closed with try-with-resources. The manager builds the JDBC URL from the database path and configures two pragmas: a busy timeout of five seconds, so a brief lock contention does not fail immediately, and foreign-key enforcement, so the reference from resource_usage to missions is honoured.

```
// db/DatabaseManager.java (openConnection, line 24)
public Connection openConnection() throws SQLException {
    Path parent = databasePath.getParent();
    if (parent == null) {
        throw new SQLException("SQLite database needs a parent directory.");
    }
    Files.createDirectories(parent);
    Connection connection = DriverManager.getConnection(jdbcUrl);
    try (Statement statement = connection.createStatement()) {
        statement.execute("PRAGMA busy_timeout = 5000");
        statement.execute("PRAGMA foreign_keys = ON");
    }
    return connection;
}
```

Listing 7.1: Opening and configuring a SQLite connection.

7.5 PreparedStatement

Every value written or queried is bound through a PreparedStatement parameter rather than concatenated into a string. This has two benefits: it prevents SQL injection through recorded values, and it lets the database reuse the compiled statement.

```
// db/SensorReadingRepository.java (findRecent, line 45, condensed)
String sql = """
        SELECT timestamp, zone, water_level, rainfall, risk_level
        FROM sensor_readings
        ORDER BY id DESC
        LIMIT ?
        """;
try (Connection connection = database.openConnection();
     PreparedStatement statement = connection.prepareStatement(sql)) {
    statement.setInt(1, limit);
    try (ResultSet results = statement.executeQuery()) {
        // map rows to immutable DatabaseHistoryEntry values
    }
}
```

Listing 7.2: A parameterized query with try-with-resources for the connection, statement, and result set.

7.6 ResultSet

A ResultSet is a cursor over the rows a query returned. Repositories read each row and map it into an immutable DatabaseHistoryEntry, so that no JDBC type leaks into the interface. The history panel then displays these entries in a table.

7.7 SQLException

SQLException is the checked exception JDBC uses to report database problems. The project never swallows it silently: DatabaseService catches it, marks the database as uninitialized so that a later operation can retry, and reports the failure to the engine, which records it in the operational journal and surfaces it in the interface.

7.8 Transactions

A transaction groups several statements so that they either all take effect or none do. This matters for mission dispatch, where three things must stay consistent: the mission row, the resource-usage rows, and the dispatch event. MissionRepository.recordDispatch performs all three on one connection with auto-commit disabled, commits them together, and rolls back if any statement fails.

```
// db/MissionRepository.java (recordDispatch, line 31, condensed)
try (Connection connection = database.openConnection()) {
    connection.setAutoCommit(false);
    try {
        // INSERT mission
        resourceUsage.insert(connection, mission.id(), resources, createdAt);
        operationalEvents.insert(connection, createdAt, "MISSION DISPATCH",
                mission.zoneId(), message);
        connection.commit();
    } catch (SQLException ex) {
        rollback(connection, ex);
        throw ex;
    }
}
```

Listing 7.3: A transaction that keeps the mission, its resources, and its event consistent.

7.9 Database Schema

The schema is created at startup by DatabaseInitializer using CREATE TABLE IF NOT EXISTS statements, followed by four indexes on the timestamp columns. Figure 3.9 in Chapter 3 shows the entity-relationship diagram; Table 7.2 lists the columns.

Table	Columns	Notes
sensor_readings	id, timestamp, zone, water_level, rainfall, risk_level	id autoincrement primary key
missions	id, mission_type, zone, status, created_at, completed_at	id is the mission identifier
resource_usage	id, resource_type, amount, mission_id, timestamp	amount CHECK (amount > 0); mission_id references missions(id)
operational_events	id, timestamp, event_type, zone, message	id autoincrement primary key

Table 7.2: SQLite schema.

7.10 Repository Layer

Each table has a repository that owns the SQL for that table. This keeps query text in one place and lets the service layer work with domain values rather than with SQL. SensorReadingRepository inserts readings and reads recent ones; MissionRepository records dispatch and completion inside transactions; ResourceUsageRepository inserts batched resource rows and reads recent allocations; OperationalEventRepository inserts and reads events.

Repository	Responsibility
SensorReadingRepository	Insert readings; read recent readings; delete readings older than the retention cutoff
MissionRepository	Transactional dispatch and completion; read recent missions
ResourceUsageRepository	Insert a mission's resource allocations; read recent allocations
OperationalEventRepository	Insert and read operational events

Table 7.3: Repository responsibilities.

7.11 Database Service

DatabaseService is the single point of contact between the simulation and the database. It owns a single-threaded executor with a bounded queue of two thousand and forty-eight tasks

and an abort policy. Every database operation is enqueued as a task, so all SQL runs on the basin-database thread and never on the Swing EDT. At startup the service initializes the schema and prunes sensor readings older than thirty days.

7.12 Asynchronous Database Operations

Because the service runs on its own thread, the response workers that call recordSensor, recordDispatch, and recordCompletion do not block on SQL. They enqueue immutable values and continue. Reads are returned as CompletableFuture instances, so the history panel can request data without freezing and update its table when the future completes on the EDT. If the queue is full, the service reports that the update was not accepted rather than blocking the caller.

7.13 Database Failure Handling

The database is optional, and the code treats it as such. If initialization fails, the service records the failure and continues to report an unavailable status; later operations retry initialization after a short delay. Crucially, a database failure never stops the simulation and never affects the file-based persistence: the journal, archive, reports, and session saves continue to work. The test suite demonstrates this directly by making the database path unusable and confirming that the simulation still processes events and that the file archive, session save, and journal are all written, with the failure recorded as a DATABASE WARNING in the journal.

CHAPTER 8: USER INTERFACE

8.1 Swing Architecture

The user interface is built with standard Swing. Swing follows a model-view-controller philosophy: components hold a model, render a view, and respond to events. BasinWatch uses this structure directly. Lists are backed by DefaultListModel instances, and the database history table is backed by a custom AbstractTableModel. The basin map is a custom component that paints itself.

8.2 Event Dispatch Thread

All Swing components are created and modified on the event-dispatch thread. The engine's workers never touch a component; instead they publish immutable snapshots, and the frame subscribes with a listener that hands each snapshot to the EDT.

```
// app/MainFrame.java (constructor, line 112)
engine.addSnapshotListener(snapshot ->
        SwingUtilities.invokeLater(() -> updateSnapshot(snapshot)));
engine.addStatusListener(message ->
        SwingUtilities.invokeLater(() -> statusLabel.setText(message)));
```

Listing 8.1: The frame marshals every engine update onto the event-dispatch thread.

This is the interface's side of the immutability contract: because the snapshot is immutable, it can be captured in a lambda and applied later on the EDT without any risk that the model has changed underneath it.

8.3 Main Operations Console

The main console is a single frame with a header, a toolbar, a central work area, and a footer. The toolbar offers the primary actions — start or pause observations, advance one interval, inject a storm cell, save a session, restore a session, and export a situation report. The central area is split horizontally between the basin map and a tabbed panel, and vertically between that work area and the operations timeline.

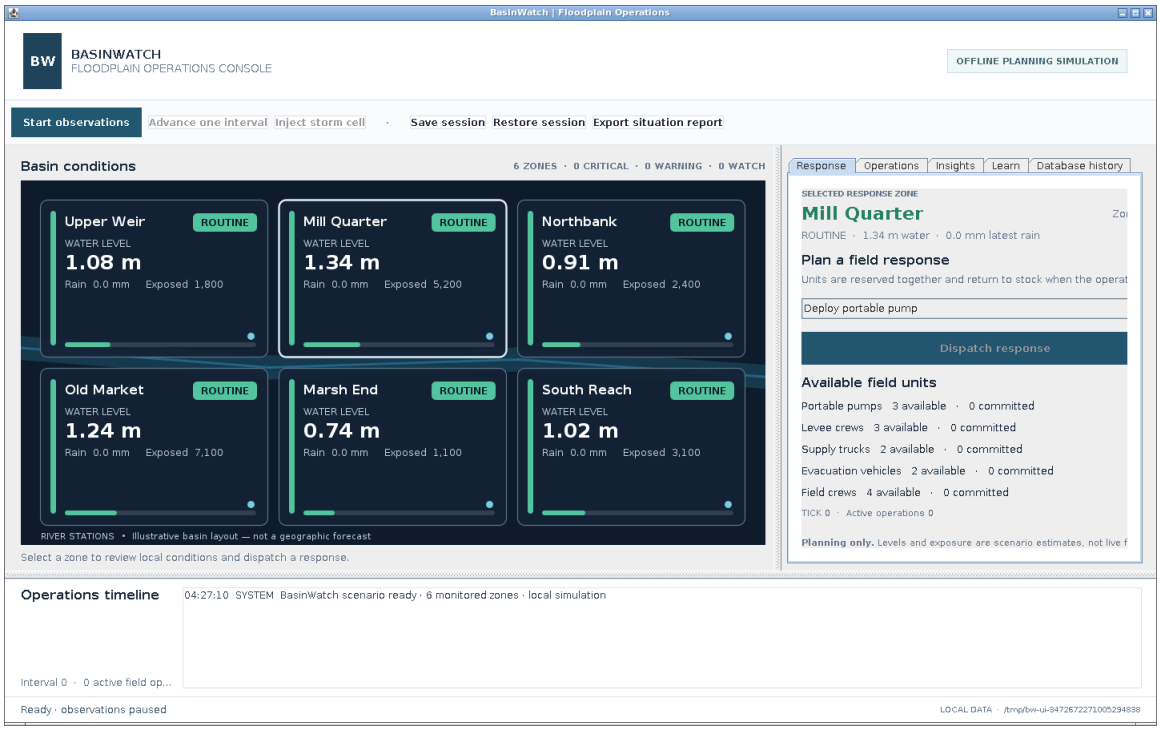


Figure 8.1: The main BasinWatch console at startup, with observations paused.

Component	Type	Responsibility
MainFrame	JFrame	Hosts the toolbar, work area, and footer; marshals updates to the EDT
BasinMapPanel	JPanel (custom paint)	Draws the schematic basin and zones; handles zone selection
DatabaseHistoryPanel	JPanel	Displays database history in a table
HistoryTableModel	AbstractTableModel	Backs the history table
MissionRenderer	DefaultListCellRenderer	Renders mission rows in the mission list

Table 8.1: Swing UI components.

Tab	Purpose
Response	Selected zone summary, mission selector, dispatch, and field-unit counts
Operations	Active and completed field missions with progress
Insights	Systems inspector: thread states, queue depth,

	processed events
Learn	In-application architecture notes
Database history	Recent records from the embedded database

Table 8.2: Console tabs and their purpose.

Control	Action
Start / Pause observations	Begin or suspend the synthetic feed
Advance one interval	Emit one interval and advance missions
Inject storm cell	Queue a marked high-rainfall reading for the selected zone
Save session	Persist the current session to a named file
Restore session	Load a saved session
Export situation report	Write a readable text report
Dispatch	Reserve resources and start the chosen mission

Table 8.3: Operator controls.

8.4 Basin Map

The basin map is a custom-painted panel. It draws a schematic river and six zone tiles, each showing the zone name, its risk category in a risk-coloured badge, its water level, its latest rainfall, and its exposed population, together with a small water-level bar. The panel is explicitly labelled as an illustrative basin layout rather than a geographic forecast. Zones are selectable by mouse click, and the panel exposes an accessible name so that the zone selector can be used for keyboard navigation.

```
// app/BasinMapPanel.java (paintComponent, line 59, condensed)
protected void paintComponent(Graphics graphics) {
    super.paintComponent(graphics);
    Graphics2D g = (Graphics2D) graphics.create();
    g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);
    paintRiver(g);
    hitTargets.clear();
    for (ZoneSnapshot zone : snapshot.zones()) {
        Rectangle bounds = /* computed tile rectangle */;
        hitTargets.put(zone.id(), bounds);
        paintZone(g, zone, bounds, zone.id().equals(selectedZone));
    }
    g.dispose();
}
```

Listing 8.2: The map panel paints the river and each zone, and records hit targets for mouse selection.

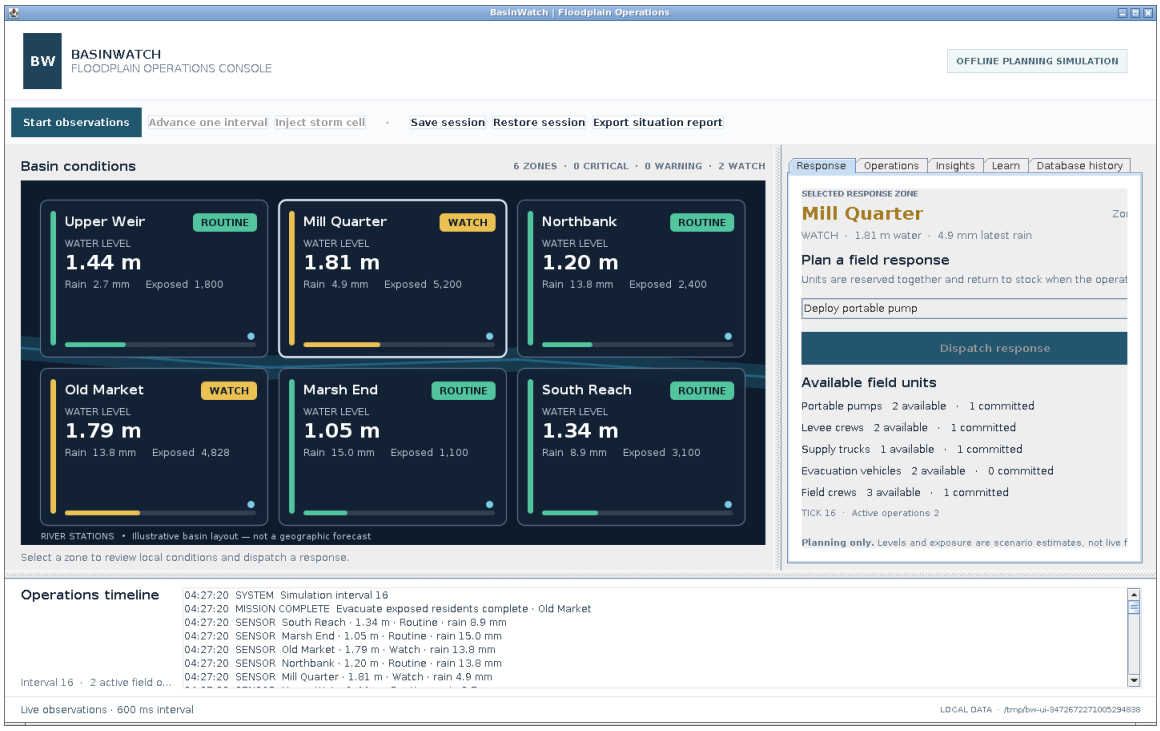


Figure 8.2: The console during a live simulation with active response missions.

8.5 Systems Inspector

The systems inspector is the "Insights" tab. It reports the state, name, and priority of every background thread, the current queue depth against its capacity, the number of processed events, and the archive backlog and waiting counts. It also hosts the optional archive-priority experiment. This view is diagnostic: the application works perfectly without it, but it makes the concurrency of the system visible, which is valuable in an educational tool.

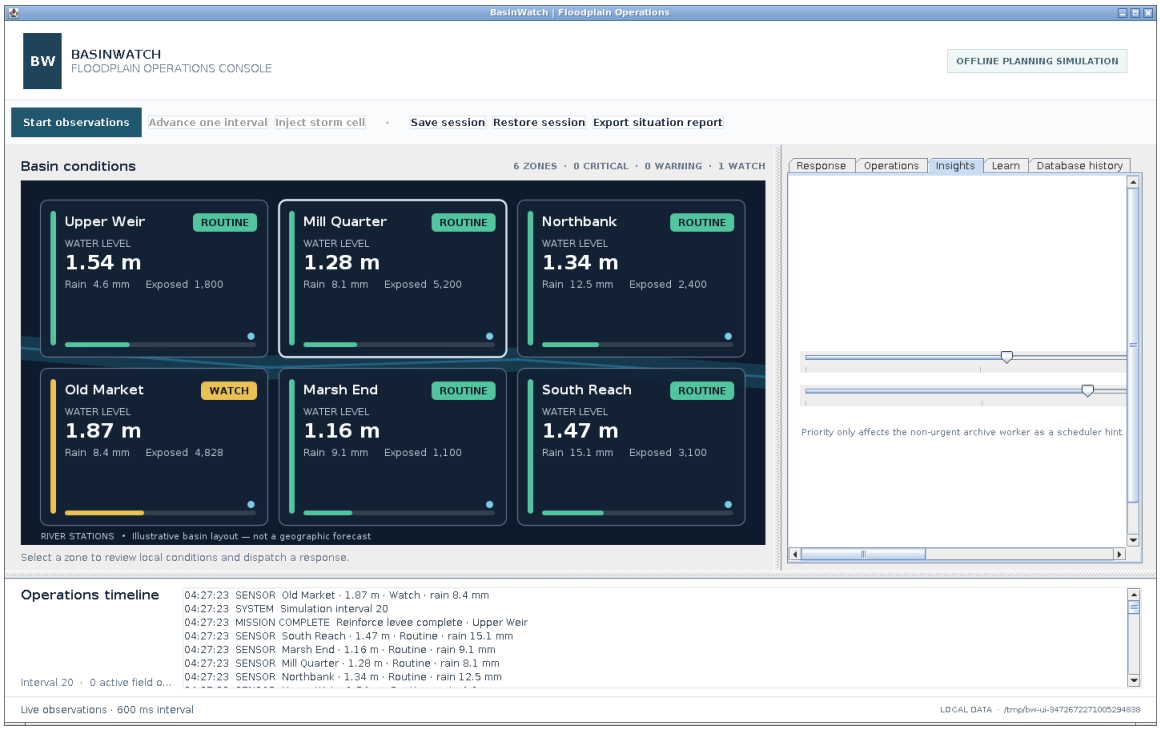


Figure 8.8: The systems inspector showing background thread states and queue metrics.

8.6 Response Desk

The "Response" tab is the response desk. It shows the selected zone's name, risk, water level, and rainfall; a mission-type selector and a dispatch button; and the available field units with their committed counts. The dispatch button is enabled only once the simulation has started. A short clarification reminds the operator that levels and exposure are scenario estimates, not live forecasts.

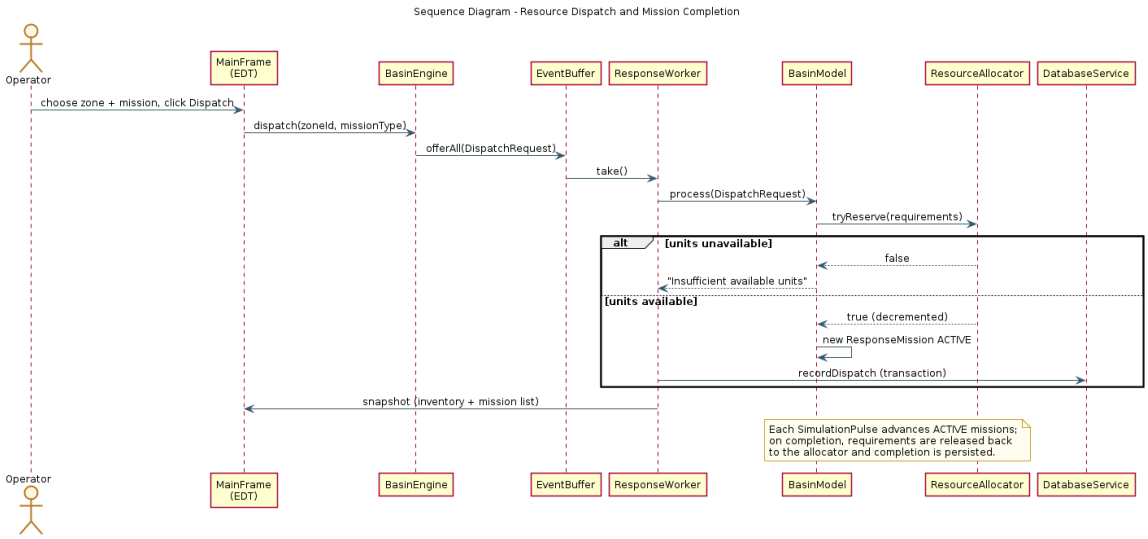


Figure 8.3: Sequence diagram of resource dispatch and mission completion.

8.7 Activity Feed

The operations timeline at the bottom of the console is the activity feed. It shows the most recent activity entries in reverse order, colour-coded by category, so the operator can follow risk updates, allocations, completions, and system events as they occur.

8.8 Database History

The "Database history" tab presents a table of recent records drawn from the SQLite database. The operator selects a category — sensor readings, missions, operational events, or resource usage — and the panel requests the data asynchronously, updating the table on the EDT when the future completes. The panel also shows the database status and the number of pending database writes.

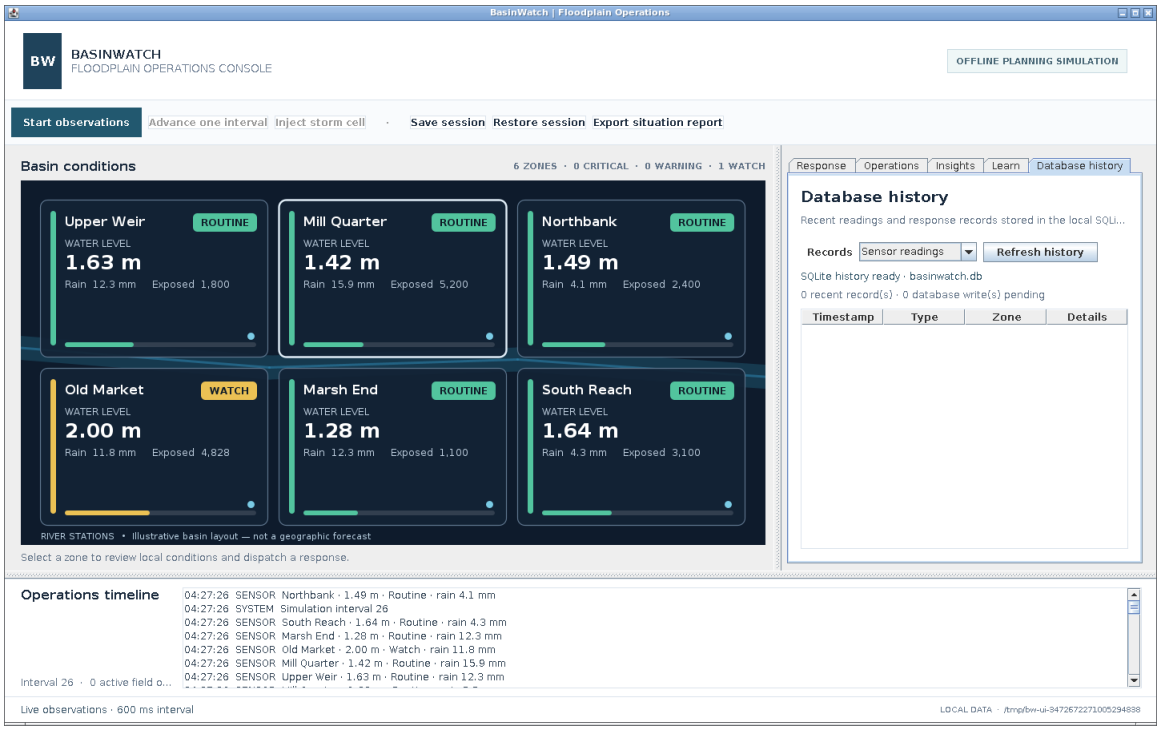


Figure 8.4: The database history tab displaying recent records.

8.9 User Interaction Flow

Figure 8.5 shows the operator's workflow: launch, restore or start fresh, start observations, read the map, select an at-risk zone, choose and dispatch a mission, follow progress, optionally inspect background activity and database history, export a report, and close cleanly.

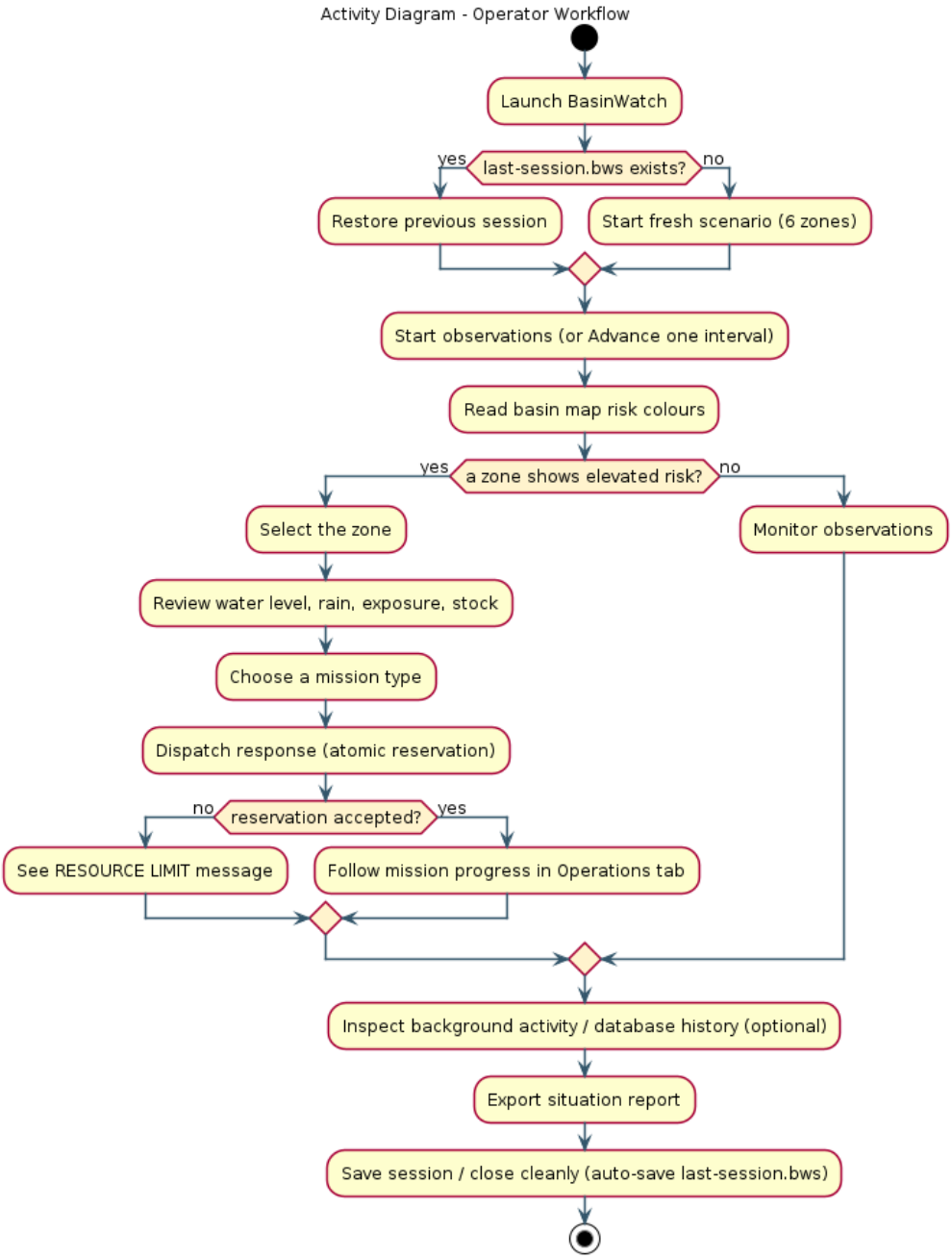


Figure 8.5: Activity diagram of the operator workflow.

8.10 UI State Updates

The interface is refreshed by a single updateSnapshot method that the EDT runs whenever a new snapshot arrives. That method updates the map, the selected-zone summary, the risk tally, the resource labels, the mission list, and the activity list. A separate timer refreshes the diagnostic labels. Because both paths run on the EDT, there is no concurrent access to any component.

8.11 Error Presentation

Errors are surfaced in two ways. Non-fatal conditions, such as a rejected dispatch, appear as a status banner and, where appropriate, a warning dialog. Failures of an asynchronous operation — a save, a restore, a report, or a close — are reported when the operation's future completes, with the underlying cause unwrapped for the operator. In every case the interface preserves the last valid state rather than leaving the display inconsistent with the model.

8.12 Screenshots

The screenshots in this chapter were produced by running the application and capturing the real interface. They are evidence of the implemented design rather than illustrations.

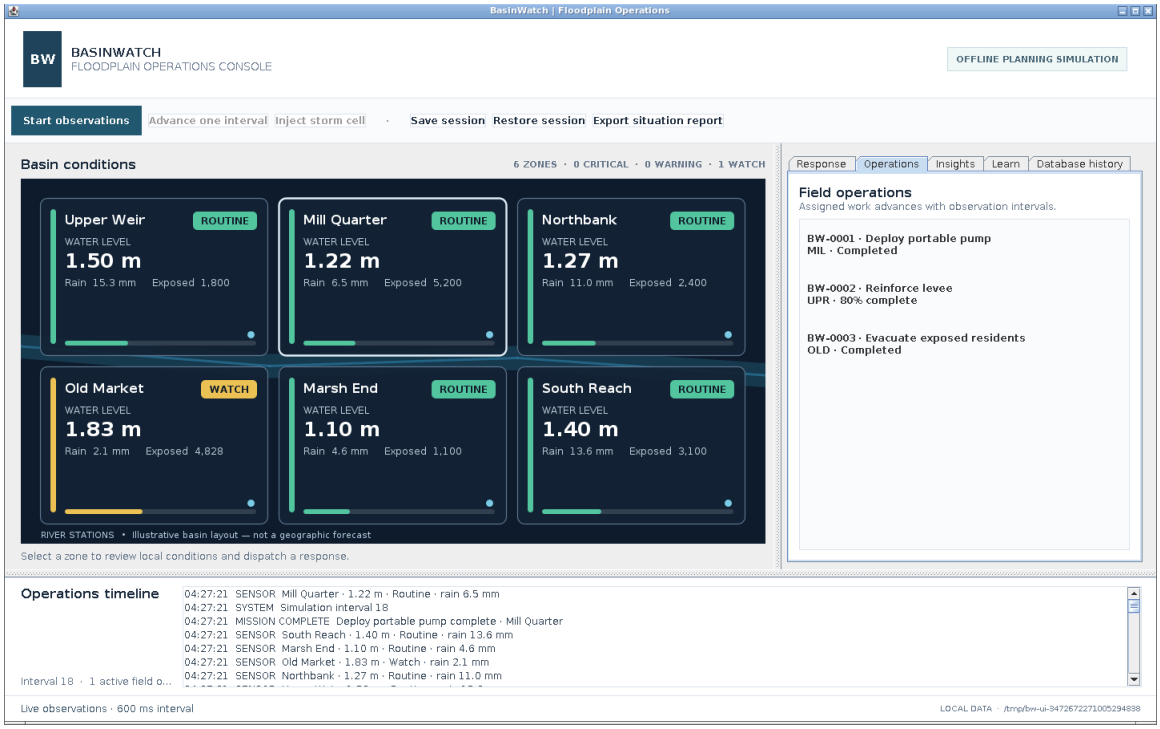


Figure 8.6: The Operations tab listing active and completed field missions.

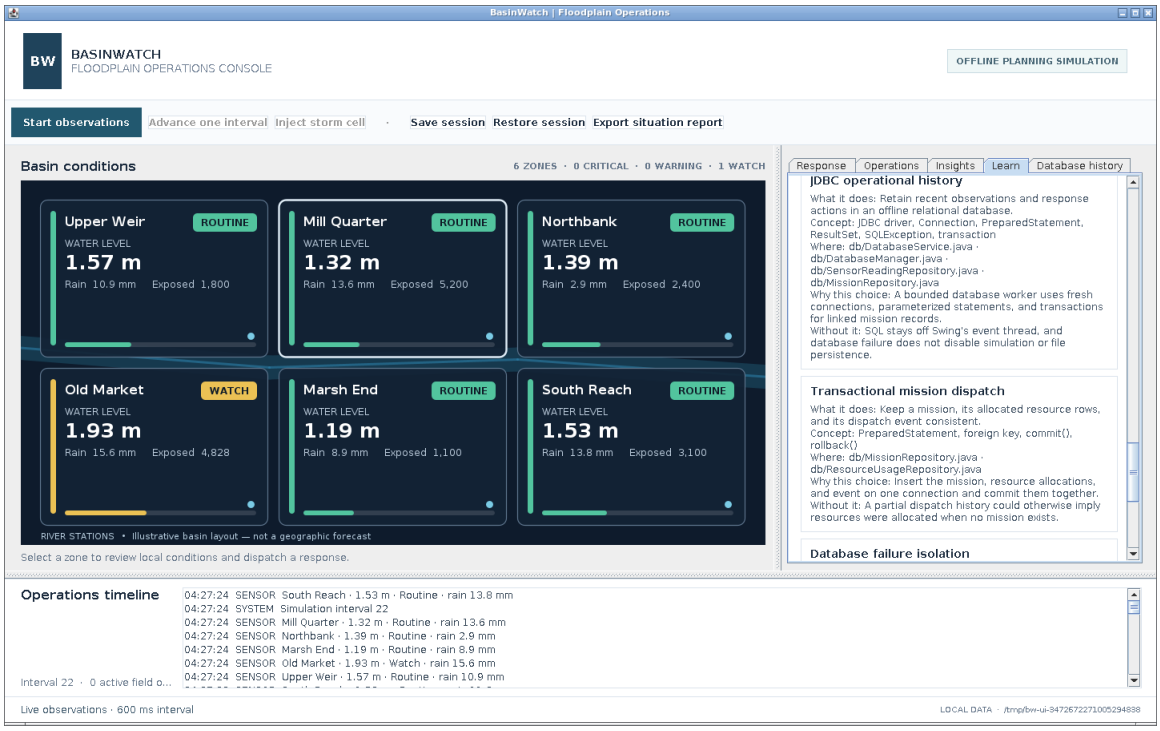


Figure 8.7: The Learn tab presenting the in-application architecture notes.

CHAPTER 9: IMPLEMENTATION

9.1 Project Structure

The project follows a conventional Maven layout. Production code lives under src/main/java in the six packages described in Chapter 3; tests live under src/test/java; helper scripts live under scripts; and the build is described by pom.xml. The application's working data is created outside the project, in the per-user data folder.

9.2 Application Startup

Startup selects the system look and feel, then schedules the real work on the EDT. It resolves the data folder, restores the last session if one exists, constructs the engine and frame, and shows the frame. If the previous session cannot be restored, the operator is asked whether to start fresh, and the saved file is retained until a later clean shutdown.

```
// app/BasinWatchApp.java (main and openApplication, condensed from line 20)
public static void main(String[] arguments) {
    try { UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); }
    catch (ReflectiveOperationException | UnsupportedLookAndFeelException ex) { /* fallback
*/ }
    SwingUtilities.invokeLater(() -> openApplication(arguments.clone()));
}
// ... in openApplication ...
paths = AppPaths.create(AppPaths.resolveDataRoot(arguments));
BasinModel model = new BasinModel();
Path lastSession = paths.saves().resolve("last-session.bws");
if (Files.exists(lastSession)) { /* load and restore, or ask to start fresh */ }
BasinEngine engine = new BasinEngine(paths, model);
MainFrame frame = new MainFrame(engine, paths);
frame.setVisible(true);
```

Listing 9.1: Startup resolves data paths, restores a session if possible, and shows the frame on the EDT.

Step	Action
1	Select the system look and feel
2	Schedule `openApplication` on the EDT
3	Resolve the data root and create data directories
4	Restore `last-session.bws` if present, or start fresh
5	Construct the `BasinEngine` (services and workers, not yet started)

6	Construct and display the `MainFrame`
7	Operator starts observations or advances a single interval

Table 9.1: Application startup sequence.

9.3 Engine Lifecycle

The engine exposes its lifecycle as `CompletableFuture`-returning methods — `start`, `pause`, `save`, `load`, `exportReport`, and `closeAsync` — all of which run on a single daemon control thread. This serialises lifecycle actions so that, for example, a save cannot run concurrently with a restore. Listing 3.1 (Chapter 3) shows the feed startup; Listing 6.5 (Chapter 6) shows shutdown.

9.4 Sensor Feed Implementation

The feed's implementation is shown in Listing 6.1. Its design points are that it uses a shared atomic tick counter, that it writes a pulse after each interval's readings, that it flushes the pipe so the reader sees complete lines, and that it sleeps interruptibly between intervals.

9.5 Sensor Intake

The intake implementation is shown in Listing 6.2. Its design points are that it reads whole lines, that it distinguishes pulses from readings, that it validates each record and discards malformed ones with a warning, and that it enqueues accepted events into the bounded buffer.

9.6 Event Processing

A worker's processing method captures the model state before and after the transition under the model lock, then performs journal writes, archive submission, and database enqueueing outside the lock. The generic processing is shown in Listing 3.2. The engine also computes, from the before-and-after snapshots, exactly which database records should be written — a sensor row when a reading was accepted and escalated or not, a dispatch record for a new mission, and a completion record when a mission's status changes to completed.

9.7 Basin Model

The model applies three kinds of event. A reading updates a zone's water level and risk; a pulse advances missions and completes those that finish; a dispatch reserves resources and creates a mission.

```
// domain/BasinModel.java (applyReading, line 61, condensed)
BasinZone zone = requireZone(reading.zoneId());
RiskLevel previousRisk = zone.snapshot().risk();
if (!zone.apply(reading)) {
    messages.add("Ignored stale station reading from " + reading.stationId());
    return;
}
tick = Math.max(tick, reading.tick());
RiskLevel currentRisk = zone.snapshot().risk();
// record a SENSOR or RISK UPDATE entry and return a message
```

Listing 9.2: Applying a reading updates the zone and detects a risk escalation.

```
// domain/BasinModel.java (applyPulse, line 79, condensed)
if (pulse.tick() <= lastPulseTick) {
    return;
}
lastPulseTick = pulse.tick();
tick = Math.max(tick, pulse.tick());
for (ResponseMission mission : missions.values()) {
    if (!mission.advance()) {
        continue;
    }
    BasinZone zone = requireZone(mission.snapshot().zoneId());
    switch (mission.snapshot().type()) {
        case PUMP_DEPLOYMENT -> zone.pump(mission.snapshot().type().waterEffectMeters());
        case LEVEE_REINFORCEMENT -> zone.reinforceLevee();
        case EVACUATION -> zone.evacuate();
    }
    resources.release(mission.snapshot().type().requirements());
    String message = mission.snapshot().type().label() + " complete · " + zone.name();
    record("MISSION COMPLETE", message);
    messages.add(message);
}
```

Listing 9.3: A pulse advances each mission, applies its effect on completion, and releases its resources.

```
// domain/BasinModel.java (applyDispatch, line 103, condensed)
BasinZone zone = requireZone(request.zoneId());
MissionType type = request.missionType();
pruneMissionHistory();
if (!resources.tryReserve(type.requirements())) {
    String message = "Insufficient available units for " + type.label();
    record("RESOURCE LIMIT", message + " · " + zone.name());
    messages.add(message);
    return;
}
String id = "BW-" + String.format("%04d", missionSequence.incrementAndGet());
```

```
ResponseMission mission = new ResponseMission(id, type, zone.id(), tick);
missions.put(id, mission);
String message = id + " dispatched · " + type.label() + " · " + zone.name();
record("UNIT ALLOCATED", message);
messages.add(message);
}
```

Listing 9.4: A dispatch reserves resources atomically and creates an active mission.

9.8 Resource Allocation

Resource allocation is implemented in ResourceAllocator (Listing 6.4). The standard stock is three pumps, three levee crews, two supply trucks, two evacuation vehicles, and four field crews. Release validates that the released quantity cannot exceed what is actually reserved, so a double release is detected rather than silently inflating the inventory.

9.9 Mission Processing

A mission advances one tick at a time and completes when its progress reaches its type's duration. The advance method is deliberately small and returns whether the mission just completed.

```
// domain/ResponseMission.java (advance, line 25)
boolean advance() {
    if (status != MissionStatus.ACTIVE) { return false; }
    progressTicks = Math.min(type.durationTicks(), progressTicks + 1);
    if (progressTicks == type.durationTicks()) {
        status = MissionStatus.COMPLETED;
        return true;
    }
    return false;
}
```

Listing 9.5: Deterministic mission progression, one tick at a time.

Step	Action	Class
1	Operator selects a zone and a mission type	MainFrame
2	Engine enqueues a DispatchRequest	BasinEngine
3	Worker reserves all requirements atomically	ResourceAllocator.tryReserve
4	Model creates an ACTIVE mission	BasinModel.applyDispatc

		h
5	Each pulse advances every ACTIVE mission by one tick	ResponseMission.advance
6	On completion, the type's effect is applied to the zone	BasinZone.pump/ reinforceLevee/evacuate
7	Requirements are released back to the inventory	ResourceAllocator.releas e
8	Completion is persisted and the snapshot updated	DatabaseService, BasinEngine

Table 9.2: Mission processing steps.

9.10 Persistence

The persistence implementations are described in Chapter 5. Each mechanism uses the stream family appropriate to its data, and each is wrapped in try-with-resources or closed explicitly during shutdown.

9.11 Database

The database implementations are described in Chapter 7. The service runs on a single daemon thread, repositories use prepared statements and transactions, and failures are isolated from the simulation.

9.12 Report Writer

The report writer produces a four-section text report (Listing 5.5 shows the structure; the implementation uses a UTF-8 buffered writer and a timestamped filename).

9.13 Session Restore

Restoring a session reads the snapshot, validates it, and rebuilds the model with fresh runtime primitives. The restore method re-checks every domain invariant: that there are exactly six zones, that resource counts are within range, that missions are valid, and that the saved stock plus the resources committed to active missions equals the standard capacity.

```
// domain/BasinModel.java (restore, line 151, condensed)
if (saved == null || saved.tick() < 0 || saved.zones().size() != 6) {
    throw new IllegalArgumentException("Saved basin state is incomplete.");
}
// validate resource counts, zones, and missions ...
```

```
for (ResourceType type : ResourceType.values()) {
    if (stock.get(type) + commitments.get(type) != standardCapacity.get(type)) {
        throw new IllegalArgumentException(
                "Saved stock and active reservations do not balance for " + type + ".");
    }
}
```

Listing 9.6: Restoring a session re-validates the domain invariants, including the resource balance.

9.14 Error Handling

Error handling is applied at every boundary, as described in Section 3.11. The recurring principles are: validate at the boundary; throw the most specific exception; report with context; never present a failure as a success; and preserve the interrupt status when a thread is interrupted.

9.15 Important Code Excerpts

The excerpts in this report are drawn verbatim from the project source and are listed with their file and line references so that the reader can locate them. They are representative rather than exhaustive; the complete source is provided with the project and, in part, in Appendix A.

CHAPTER 10: TESTING AND VALIDATION

10.1 Testing Strategy

The project uses a dependency-free assertion suite rather than an external test framework. The suite is a plain Java program with a main method, compiled by Maven and executed with assertions enabled and the SQLite JDBC driver on its classpath by the scripts/test.ps1 runner. This choice keeps the test infrastructure minimal and transparent: there is no framework to configure, and the whole suite can be read and understood directly.

The strategy is to test *invariants* rather than incidental behaviour. An invariant is a property that must hold no matter how threads are scheduled: a reservation must never over-allocate; a full buffer must never lose an accepted event; a restored session must match the saved one; a corrupt save must never be accepted. Testing invariants is the right approach for a concurrent system, because it is exactly the properties that matter under concurrency that are hardest to observe by hand.

The suite defines three helpers: check for a boolean condition, expect for an expected exception type, and await for a condition that should become true within a timeout. Each helper increments an assertion counter. When the suite was executed during the preparation of this report, it printed:

```
BasinWatch invariant suite passed (76 assertions).
```

Listing 10.1: The result reported by the invariant suite when executed.

10.2 Unit and Invariant Tests

The first suite, parsesAndValidatesSensorEvents, checks that a sensor reading survives an encode-and-parse round trip unchanged, and that malformed input is rejected: a line with a non-numeric field, a negative tick, and an identifier containing an illegal character each raise IllegalArgumentException.

10.3 Concurrency Tests

Concurrency is tested through the buffer suite, coordinatesBoundedEventBuffer, which uses real threads and latches. It verifies that an empty consumer waits on the queue condition, that a producer wakes it with a real event, that a full bounded queue applies back-pressure to

a producer, that removing an item releases the waiting producer, and that closing the queue wakes waiters with an explicit end-of-stream result.

10.4 Resource Contention Tests

The suite `reservesSharedResourcesAtomically` is the clearest demonstration of the resource invariant. Twenty-four threads are released simultaneously from a start gate, each attempting to reserve the last pump and the last field crew. The suite asserts that **exactly one** reservation succeeds, that the reserved pump becomes unavailable, that a release returns it to stock, that a second release of the same reservation raises `IllegalStateException`, and that the rejected duplicate release does not corrupt the crew inventory.

10.5 Buffer Tests

Buffer behaviour is covered by the concurrency suite described in Section 10.3. The buffer's own counters (`waitingConsumers`, `waitingProducers`, `activeConsumers`) make the conditions observable, which is why the suite can wait for a specific condition rather than guessing at a delay.

10.6 Persistence Tests

Persistence is covered by two suites. `validatesArchiveHeaders` writes two records to a binary archive, reopens the archive to append a third, and confirms that a file with an invalid header is rejected with `IOException`. `savesRestoresAndRejectsUnsafeSnapshots` saves a model with a reading and an active evacuation, restores it, and confirms that the tick, the zone measurements, the active mission, and the resource inventory all round-trip exactly.

10.7 Database Tests

`persistsJdbcHistoryAndTransactions` writes a sensor reading, dispatches a mission, and completes it, then reads all four history categories back. It asserts that the sensor row contains the expected rainfall, that the mission row shows completion, that every resource allocated to the mission was recorded, that exactly two events were committed for the mission, that a parameterized retention delete removes the sensor row, and that no database failures were reported during successful operation.

10.8 Session Round-Trip Tests

The snapshot suite and the engine suite together cover session round-tripping. The engine suite saves a live session, confirms the file reaches disk and that a save made during live observations resumes the feed afterwards, then restores the session and confirms that the loaded tick matches the saved tick and that a persisted mission completes through the existing simulation.

10.9 Failure Tests

Failure handling is tested directly. The snapshot suite rejects a four-byte corrupt file and a serialized ArrayList that is not a BasinWatch session. The database-unavailable suite makes the database path unusable and confirms that the simulation still processes events, that the binary archive and the automatic session save are still written, and that the operational journal records a DATABASE WARNING.

10.10 Shutdown Tests

The engine suite closes the engine and confirms that a clean shutdown automatically saves last-session.bws, that the operational journal is non-empty, and that the binary archive contains its format marker, its version, and at least one typed record.

10.11 UI Testing

The user interface was validated by running the application and exercising its controls, and the screenshots in this report are the evidence. Automated GUI testing was not implemented; the interface is a thin layer over the tested engine, and its correctness depends chiefly on the engine's snapshot and status listeners, which the engine suite exercises.

10.12 Test Case Table

Test ID	Component	Scenario	Expected result	Verified result	Status
T-01	SensorReading	Encode and parse a reading	Round-trips exactly	Round-trips exactly	Pass
T-02	SensorReading	Malformed / out-of-range input	IllegalArgumentException	Thrown	Pass

T-03	ResourceAllocator	24 threads reserve the last pump+crew	Exactly one succeeds	Exactly one succeeds	Pass
T-04	ResourceAllocator	Release then double release	Stock restored; second release throws	IllegalStateException	Pass
T-05	EventBuffer	Empty consumer waits	Consumer waits; producer wakes it	Verified via counters	Pass
T-06	EventBuffer	Full bounded queue	Producer waits (back-pressure)	Verified via counters	Pass
T-07	EventBuffer	Close with a waiting consumer	Waiter woken with end-of-stream	Verified	Pass
T-08	BasinModel	Dispatch pump; advance to completion	Water falls; pump released; mission completed	Verified	Pass
T-09	BasinModel	Levee reinforcement effect	Subsequent runoff reduced	Verified	Pass
T-10	SnapshotStore	Save and restore a session	State round-trips exactly	Verified	Pass
T-11	SnapshotStore	Corrupt / foreign file	IOException	Thrown	Pass
T-12	SensorArchive	Valid and invalid headers	Valid appends; invalid rejected	Verified	Pass
T-13	AppPaths	Data-dir resolution	Valid accepted; invalid rejected	Verified	Pass
T-14	DatabaseService	JDBC insert/select/tra	Rows and transactions	Verified	Pass

		nsaction	correct		
T-15	DatabaseService	Retention delete	Old rows removed	Verified	Pass
T-16	BasinEngine	Full offline session	Events processed; save/restore/report/journal/archive correct	Verified	Pass
T-17	BasinEngine	Database unavailable	Simulation and file persistence continue	Verified	Pass
T-18	BasinEngine	Clean shutdown	Auto-save written; journal non-empty	Verified	Pass

Table 10.1: Test-case summary. Results reflect the executed run of the invariant suite.

Suite	Focus	Assertions contributed
parsesAndValidatesSensorEvents	Event parse and validation	Round-trip and rejection checks
reservesSharedResourcesAtomically	Resource contention	Atomic reservation and release checks
coordinatesBoundedEventBuffer	Bounded buffer	Waiting, back-pressure, close checks
advancesAndReleasesResponseMissions	Mission progression	Advance, effect, release checks
savesRestoresAndRejectsUnsafeSnapshots	Session round-trip	Save/restore and rejection checks
validatesArchiveHeaders	Binary archive	Header and append checks
resolvesPerUserDataLocations	Data-dir resolution	Accept and reject checks
persistsJdbcHistoryAndTransactions	JDBC history	Insert, select, transaction, retention checks
runsACompleteOfflineEngineSession	Full engine session	Feed, save/restore, report, journal, archive checks
continuesWhenDatabaseI	Failure isolation	Continue-on-failure checks

sUnavailable		

Table 10.2: Executed evidence summary. The suite reported seventy-six passing assertions in total.

10.13 Results

The invariant suite executed successfully and reported seventy-six passing assertions across its ten suites. Implemented test coverage includes concurrent resource contention, bounded-buffer waiting and closing, event validation and processing, mission progression and release, session round-trip and corrupt-input rejection, binary-archive validation, data-directory resolution, JDBC history and transactions, database-unavailable resilience, and orderly shutdown. No performance figures are claimed, because the suite asserts correctness rather than speed.

10.14 Observed Limitations

The suite does not include automated GUI tests, and it does not measure throughput or latency. The resource-contention test uses a small, deterministic scenario rather than a long randomised stress run. These limitations are stated plainly rather than hidden; they are reasonable for a teaching project whose aim is to demonstrate correct concurrency rather than to certify performance.

CHAPTER 11: RESULTS AND DISCUSSION

11.1 Functional Results

Every function required in Chapter 2 was observed to work in the running application. The application starts, presents six zones, accepts a live observation feed or single-interval steps, allows a storm cell to be injected, validates observations, ranks and highlights risk, maintains a finite inventory, dispatches three mission types, reserves resources atomically, advances and completes missions, returns resources on completion, displays inventory and active operations, writes a journal, archives typed measurements, saves and restores sessions, exports a report, and maintains an optional database history. The screenshots in Chapters 6 and 8 are direct evidence of the interface.

11.2 Runtime Workflow

The runtime workflow behaved as designed. The sensor feed produced observations on its interval, the intake task validated and enqueued them, the response workers applied them to the model, and the interface updated on the EDT from immutable snapshots. During live observation the interface remained responsive to selection and dispatch, which is the practical demonstration that the pipeline does not block the UI.

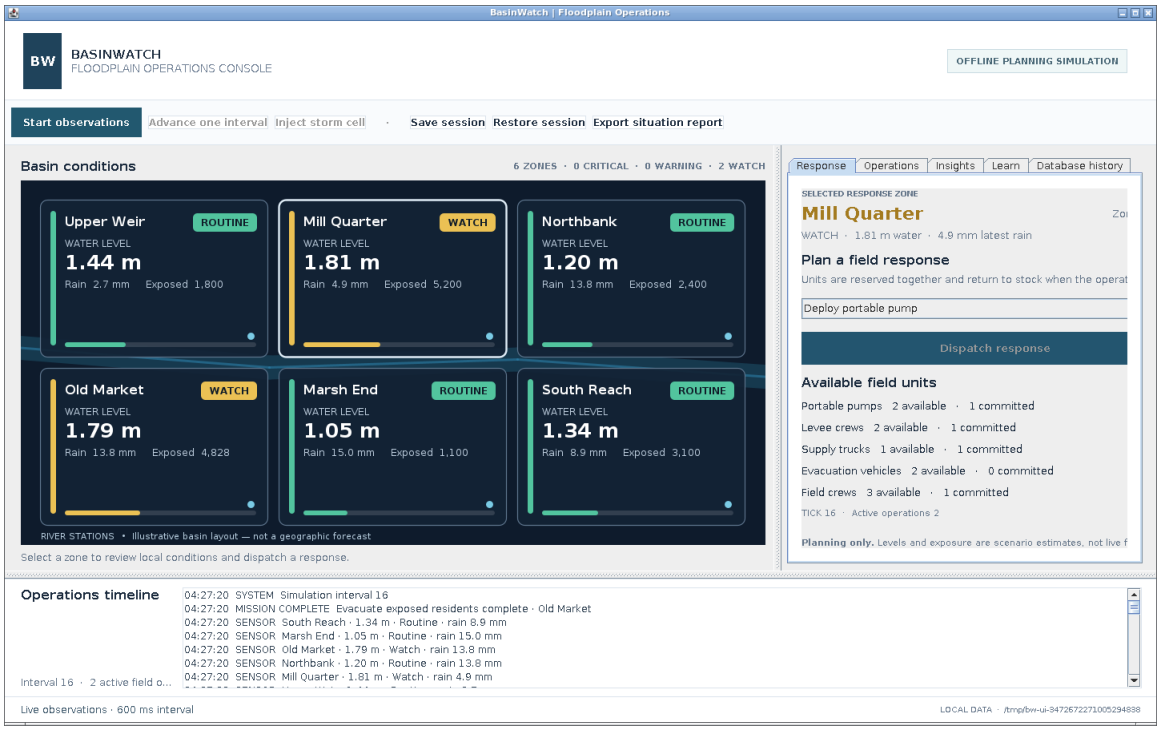


Figure 11.1: The basin map and operations timeline during a live run.

11.3 Resource Allocation Results

Resource allocation was verified both by the test suite and by observation. In the suite, twenty-four simultaneous contenders produced exactly one successful reservation of the final pump and crew, and a duplicate release was rejected without corrupting inventory. In the running application, dispatching missions reduced the available counts and committing units to active operations, and completing a mission returned the units to the available stock. No double allocation was observed.

11.4 Persistence Results

All five persistence mechanisms worked. The operational journal accumulated readable timestamped entries; the binary archive was created with its marker and version and contained typed records; the session save wrote a file that could be restored; the report writer produced a multi-section text report; and the session's automatic save on clean exit produced a resumable last-session.bws. A representative report generated during verification ran to several kilobytes and contained zone conditions, available resources, response operations, and recent activity.

11.5 Database Results

The optional database behaved as designed. During verification, the history accumulated typed rows across all four tables, and the history panel displayed them. The transaction test confirmed that a mission, its resource rows, and its dispatch event are committed together, and the failure test confirmed that an unusable database does not stop the simulation or the file-based persistence.

11.6 Concurrency Results

Concurrency results were positive and, importantly, *bounded*. The buffer applied back-pressure rather than growing without limit; the allocator never over-allocated under contention; snapshots were delivered to the EDT without any observed inconsistency; and shutdown drained accepted work and closed files within the bounded timeout. Thread states reported by the inspector were the platform's own truthful states, and the optional archive-priority experiment changed a scheduling hint without affecting correctness.

11.7 OOP Learning Outcomes

The project produced concrete learning outcomes that map directly to the syllabus. Encapsulation was used to make invalid domain states unrepresentable. Abstraction through an event interface kept the pipeline stable. Narrow inheritance was used only where a framework contract required it. Polymorphism localised the differences between events, missions, and risks. Composition gave the model a single owner for state that must remain consistent. Enums replaced error-prone string categories. Immutability was the mechanism that made concurrent reads safe. Exception handling kept failures visible and local. Serialization and JDBC demonstrated persistence through two very different APIs. In short, the object-oriented concepts were not studied in the abstract; they were the tools that solved the domain's problems.

11.8 Requirement Achievement

Objective	Target	Outcome	Status
O1 Concurrent pipeline	Producer-consumer with responsive UI	Implemented and exercised	Achieved
O2 Atomic allocation	All-or-nothing	Verified (1 of 24	Achieved

	reservation	succeeds)	
O3 Bounded buffering	Bounded buffer with back-pressure and drain	Verified	Achieved
O4 Multi-format persistence	Journal, archive, snapshot, report	All four implemented	Achieved
O5 Safe session recovery	Save/restore with corrupt-input rejection	Verified	Achieved
O6 Optional relational history	SQLite via JDBC, isolated failure	Verified	Achieved
O7 Syllabus demonstration	Core OOP/Java concepts applied	Documented in Chapter 4	Achieved

Table 11.1: Requirement achievement. Status reflects verified functionality, not measured performance.

Observation	Evidence
Application starts and shows the console	Screenshot Figure 8.1
Live simulation with active missions	Screenshot Figure 8.2
Zone risk and water levels update	Screenshots Figures 8.2 and 11.1
Operations list shows mission progress	Screenshot Figure 8.6
Systems inspector reports thread states and queue metrics	Screenshot Figure 8.8
Database history table populated	Screenshot Figure 8.4
Session save and restore work	Invariant suite (T-10, T-16)
Situation report generated	Report writer (Chapter 5); Appendix B
Atomic reservation under contention	Invariant suite (T-03)

Table 11.2: Verified runtime observations and their evidence.

11.9 Discussion

Two observations deserve emphasis. First, the design's safety comes from *structure*, not from luck. The single inventory monitor, the bounded buffer, and immutable snapshots each remove a specific class of bug by construction. The project does not rely on threads happening to run in a convenient order, and it does not treat priority as a correctness mechanism. Second, the simulation's honesty is itself a design feature. By labelling the map as schematic, the parameters as illustrative, and the application as an offline planning tool, the project avoids the trap of presenting a teaching artefact as an operational system. The

result is a program that is both instructive about object-oriented programming and candid about its own limits.

CHAPTER 12: CONCLUSION AND FUTURE SCOPE

12.1 Conclusion

BasinWatch demonstrates that a single, offline Java desktop application can model a genuinely concurrent coordination problem and solve it with disciplined object-oriented design. A bounded producer-consumer pipeline keeps the interface responsive while observations and operations proceed independently; a short synchronized transaction makes resource reservation atomic; immutable snapshots make the hand-off to the interface safe; and a family of persistence mechanisms — text journal, binary archive, session snapshot, readable report, and optional embedded database — preserves the run in forms suited to human review, typed replay, and structured query. The implementation is supported by an executed assertion suite that reported seventy-six passing assertions across concurrency, resource contention, buffering, persistence, database, session round-trip, failure, and shutdown scenarios. Every mechanism described in this report is drawn from the actual source, and every limitation is stated rather than hidden.

12.2 Achievements

The project achieved a complete, runnable, tested simulation with a clean package structure, a documented architecture, a demonstrated concurrency model, a safe and versioned persistence design, an optional relational history, and a readable interface. It produced a substantial body of original code and a matching test suite, and it connected each part of that work to the object-oriented syllabus.

12.3 Current Limitations

The basin map is schematic and not geospatially accurate. The simulation parameters are illustrative, not calibrated hydrology. The application is single-user and offline, does not ingest real sensors, does not issue alerts, and controls no real infrastructure. Automated GUI testing was not implemented. Performance is not measured. The session format is application-private and versioned.

12.4 Future Improvements

Several improvements follow directly from the repository's own stated future direction and from the design: importing a user-authored scenario format; adding replay scrubbing; exporting CSV reports; improving map editing; adding explicit fault-injection scenarios for resource contention; and validating a hydrology model before any real-world advisory use. Each is a natural extension of the existing architecture rather than a rewrite.

12.5 Possible Extensions

Extension	Description	Technology
Scenario import	Load a user-defined basin scenario	Character streams + validation
Replay scrubbing	Step backward and forward through a recorded run	Binary archive + snapshot replay
CSV export	Export history and reports as CSV	Character streams
Map editing	Let the operator place zones and stations	Swing custom painting
Fault injection	Deliberately contend resources to demonstrate failure	Test harness + allocator
Hydrology validation	Replace illustrative parameters with a validated model	Numerical modelling (out of current scope)

Table 12.1: Possible future extensions.

REFERENCES

1. Oracle Corporation. *Java Platform, Standard Edition 17 API Specification*. Available: https://docs.oracle.com/en/java/javase/17/docs/api/

2. Oracle Corporation. *The Java Tutorials: Concurrency*. Available: https://docs.oracle.com/javase/tutorial/essential/concurrency/

3. Oracle Corporation. *The Java Tutorials: Basic I/O*. Available: https://docs.oracle.com/javase/tutorial/essential/io/

4. Oracle Corporation. *Java Object Serialization Specification*. Available: https://docs.oracle.com/en/java/javase/17/docs/specs/serialization/

5. Oracle Corporation. *Interface ObjectInputFilter (Java SE 17)*. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/io/ObjectInputFilter.html

6. Oracle Corporation. *Class Thread (Java SE 17)*. Available: https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Thread.html

7. Oracle Corporation. *The Java Tutorials: Creating a GUI With Swing*. Available: https://docs.oracle.com/javase/tutorial/uiswing/

8. Oracle Corporation. *The Java Tutorials: JDBC Database Access*. Available: https://docs.oracle.com/javase/tutorial/jdbc/

9. Apache Software Foundation. *Apache Maven Project Documentation*. Available: https://maven.apache.org/

10. SQLite Consortium. *SQLite Documentation*. Available: https://www.sqlite.org/docs.html

11. Xerial Project. *sqlite-jdbc: SQLite JDBC Driver*. Available: https://github.com/xerial/sqlite-jdbc

12. Bloch, J. *Effective Java*, 3rd ed. Addison-Wesley, 2018.

13. Horstmann, C. S. *Core Java, Volume I — Fundamentals*, 11th ed. Pearson, 2018.

14. Goetz, B. et al. *Java Concurrency in Practice*. Addison-Wesley, 2006.

APPENDIX A — IMPORTANT SOURCE CODE

The following excerpts are reproduced verbatim from the project. Line numbers refer to the repository revision used for this report.

A.1 Application startup — app/BasinWatchApp.java (lines 20–61)

A.2 Engine feed startup — engine/BasinEngine.java (lines 279–303)

A.3 Engine event processing — engine/BasinEngine.java (lines 441–472)

A.4 Engine shutdown — engine/BasinEngine.java (lines 340–417)

A.5 Sensor feed — engine/SensorFeed.java (lines 33–72)

A.6 Sensor intake — engine/SensorIntake.java (lines 26–50)

A.7 Event buffer — engine/EventBuffer.java (lines 23–84)

A.8 Resource allocator — domain/ResourceAllocator.java (lines 27–64)

A.9 Basin model dispatch — domain/BasinModel.java (lines 103–125)

A.10 Sensor archive — io/SensorArchive.java (lines 19–51)

A.11 Snapshot store — io/SnapshotStore.java (lines 23–122)

A.12 Operational journal — io/OperationalJournal.java (lines 18–41)

A.13 JDBC dispatch transaction — db/MissionRepository.java (lines 31–54)

A.14 Parameterized query — db/SensorReadingRepository.java (lines 45–71)

APPENDIX B — TEST RESULTS

The invariant suite was executed with assertions enabled and the SQLite JDBC driver on the classpath. It reported:

```
BasinWatch invariant suite passed (76 assertions).
```

The ten suites are: parsesAndValidatesSensorEvents, reservesSharedResourcesAtomically, coordinatesBoundedEventBuffer, advancesAndReleasesResponseMissions,

savesRestoresAndRejectsUnsafeSnapshots, validatesArchiveHeaders, resolvesPerUserDataLocations, persistsJdbcHistoryAndTransactions, runsACompleteOfflineEngineSession, and continuesWhenDatabaseIsUnavailable.

APPENDIX C — DATABASE SCHEMA

```
CREATE TABLE IF NOT EXISTS sensor_readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    zone TEXT NOT NULL,
    water_level REAL NOT NULL,
    rainfall REAL NOT NULL,
    risk_level TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS missions (
    id TEXT PRIMARY KEY,
    mission_type TEXT NOT NULL,
    zone TEXT NOT NULL,
    status TEXT NOT NULL,
    created_at TEXT NOT NULL,
    completed_at TEXT
);
CREATE TABLE IF NOT EXISTS resource_usage (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    resource_type TEXT NOT NULL,
    amount INTEGER NOT NULL CHECK (amount > 0),
    mission_id TEXT NOT NULL REFERENCES missions(id),
    timestamp TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS operational_events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT NOT NULL,
    event_type TEXT NOT NULL,
    zone TEXT,
    message TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_sensor_readings_time ON sensor_readings(timestamp
DESC);
CREATE INDEX IF NOT EXISTS idx_missions_created ON missions(created_at DESC);
CREATE INDEX IF NOT EXISTS idx_events_time ON operational_events(timestamp DESC);
CREATE INDEX IF NOT EXISTS idx_resource_usage_time ON resource_usage(timestamp
DESC);
```

APPENDIX D — CONFIGURATION AND BUILD INFORMATION

Build. The project targets Java 17 and is built with Maven. The only runtime dependency is org.xerial:sqlite-jdbc. The Maven Shade plugin bundles the driver and sets the main class so that a single executable JAR results.

```
mvn package
java -jar target/basinwatch-1.0.0.jar
```

Data locations. On Windows, %APPDATA%\BasinWatch\data\; elsewhere, ~/.basinwatch/data/. Subfolders: logs, archive, saves, reports, and the file basinwatch.db. The location can be overridden with --data-dir <folder>.

Default parameters. Observation interval 1500 ms (range 350–8000); storm intensity 1.0 (range 0.2–2.5); event-buffer capacity 96; three response workers; save-size limit 5 MB; database queue 2048; history limit 200; sensor retention 30 days.

APPENDIX E — ADDITIONAL SCREENSHOTS

Screenshots of the application at startup, during live operation, and across its tabs are included in Chapters 6 and 8. They were captured from the running application.

APPENDIX F — ADDITIONAL DIAGRAMS

The editable PlantUML sources for every diagram in this report are retained with the project so that the diagrams can be regenerated or amended. They cover the system context, use case, high-level architecture, package structure, data flow, runtime workflow, domain and engine class structure, thread map, persistence architecture, database ER model, session flow, thread lifecycle, producer-consumer buffer, resource allocation, application lifecycle, JDBC architecture, sequence diagrams for the sensor pipeline and resource dispatch, the operator activity diagram, and the OOP concept map.